Contents

1 Introduction **4**

2 Related Work **5**
2.1 Sparse Mixture of Experts . . . 5
2.2 Shared Experts and Structured Routing . . . 5
2.3 Parameter-Efficient Transfer Learning . . . 6
2.4 Router Stability and Noise Injection . . . 6
2.5 Fused Kernels and Systems Optimizations . . . 6

3 Model Architecture **6**
3.1 AS-MoE as a Drop-In Replacement for the FFN Layer . . . 6
3.2 The Expert Grid: A Three-Dimensional Structure . . . 7
3.3 Inside a Single Expert: Core + LoRA . . . 8
3.4 The Forward Pass: Step by Step . . . 8
3.5 Integration into a Transformer . . . 10
3.6 Concrete Model Configurations . . . 10
3.7 Training FLOP Budget Analysis . . . 12

4 Expert Topology **12**
4.1 The Three Axes of Structure . . . 12
4.2 Generation: Hierarchical Budget . . . 12
4.3 Flavor: Functional Specialization . . . 13
4.4 Channel: Multi-View Representation . . . 13
4.5 Topological Invariants . . . 13

5 Parameter Factorization: Core + LoRA **14**
5.1 Factorization Structure . . . 14
5.2 Initialization: Kaiming with Scaling . . . 14
5.3 Kato Perturbation Theory Interpretation . . . 14
5.4 Spontaneous Symmetry Breaking in Core-LoRA Dynamics . . . 15
5.5 Transfer Learning Protocol . . . 16

6 Multi-Channel Routing **16**
6.1 Generation Routing: Consensus . . . 16
6.2 Flavor Routing: Diversification . . . 17
6.3 Hybrid Routing: Expert Choice + Token Choice . . . 18

7 Budget Control via Lagrangian Duality **18**
7.1 Problem Formulation . . . 18
7.2 Lagrangian Dual Variable with Two-Phase Update . . . 18

8 Covariance Gradient Mixing **19**
8.1 The Expert Isolation Problem . . . 19
8.2 Gradient Mixing Mechanism . . . 19
8.3 Gauge-Theoretic Perspective on Gradient Confinement . . . 20
8.4 Convergence Guarantee . . . 21
8.5 Memory-Efficient Implementation . . . 21

9 Router Stability: GARN **22**
9.1 Router Collapse as Dynamical System . . . 22
9.2 Gradient-Adaptive Router Noise . . . 22
9.3 Stability Analysis . . . 23
9.4 Fokker–Planck Analysis and Stationary Distribution . . . 23

10 Path Consensus and Cross-Layer Coherence **24**
10.1 Motivation . . . 24
10.2 Path Consensus Loss . . . 24
10.3 Soft Constraint via Lagrangian . . . 24
10.4 Relationship to Curriculum Logic . . . 24

11 Mathematical Foundations **25**
11.1 Master Functional Equation . . . 25
11.2 Convergence of the Full Objective . . . 25
11.3 Operator-Theoretic View: Core + LoRA as Perturbation . . . 26
11.4 Spectral Gap and Optimal LoRA Rank Constraint . . . 26
11.5 Algorithmic Formalization . . . 27

12 Ontogenic Pre-training and Category-Theoretic Bridge **29**
12.1 From "Zero Quarks" to Language . . . 29
12.2 Phase 0: Synthetic Quarks . . . 30
12.3 Phase 1: Language Immersion . . . 30
12.4 Category-Theoretic Formalization . . . 30
12.5 Domain Transfer as a Natural Transformation . . . 31
12.6 Sample Complexity of the Compositional Functor . . . 31

13 Systems Design: CUDA and Data-Oriented Design **32**
13.1 Data-Oriented Design Principles . . . 32
13.2 Fused Grouped GEMM . . . 32
13.3 Persistent Kernels: FlashDMoE Integration . . . 33
13.4 Fused Computation-Communication via NVSHMEM . . . 33

14 Training Methodology **34**
14.1 Three-Phase Training Protocol . . . 34
14.1.1 Phase 0: Ontogenic Pre-training (Quark-Level) . . . 34
14.1.2 Phase 1: Language Immersion . . . 34
14.1.3 Phase 2: Domain Transfer (Optional) . . . 35
14.2 Anti-Forgetting: Activation-Importance Consolidation (AIC) . . . 35
14.3 Routing Replay Buffer . . . 35
14.4 Training Recipe: Hyperparameters and Schedules . . . 36
14.4.1 Component Enable/Disable Schedule . . . 36
14.5 Initialization Recipe . . . 36
14.6 Training Pipeline: Visual Overview . . . 39
14.7 Training Loop: Complete Pseudocode . . . 39
14.8 Debugging Guide: Common Failure Modes and Remedies . . . 41
14.8.1 Routing Collapse . . . 41
14.8.2 Budget Drift . . . 41
14.8.3 Expert Stagnation (No Specialization) . . . 41
14.8.4 Gradient Mixing Destabilizes Training . . . 42
14.8.5 Channel Collapse . . . 42

15 Reference Implementation **42**
15.1 GradientMixing (Dimension-Preserving, Per-Channel) . . . 42
15.2 ASMoELayer (Vectorized Forward) . . . 43
15.3 Routing Replay Buffer . . . 47

16 Experimental Design **47**
16.1 Datasets and Benchmarks . . . 47
16.2 Proposed Benchmarks . . . 48
16.3 Routing Stability and Utilization Metrics . . . 48
16.4 Ablation Studies . . . 49

17 Discussion **49**
17.1 From Quark Analogy to Standard Model: From Metaphor to Method . . . 49
17.2 Known Implementation Issues . . . 50
17.3 Limitations and Open Questions . . . 51

18 Conclusion **52**

A Engineering PRD: AS-MoE Production System **52**
A.1 Overview . . . 52
A.2 System Requirements . . . 52
A.3 Architecture Components . . . 52
A.3.1 Training Pipeline . . . 52
A.3.2 Inference Pipeline . . . 52
A.4 Configuration Schema . . . 54
A.5 Milestones . . . 54

B Notation Reference **55**

1 Introduction

Mixture of Experts (MoE) architectures have emerged as the dominant paradigm for scaling transformer-based language models beyond the computational constraints of dense networks. By activating only a sparse subset of parameters per token, MoE models achieve parameter counts in the trillions while maintaining inference costs comparable to dense models orders of magnitude smaller. However, the fundamental design of MoE layers—particularly the questions of *how experts should be organized*, *how tokens should be routed*, and *how expert specialization should be maintained during training*—remains largely ad hoc. Current architectures typically treat experts as an unstructured bag of interchangeable feed-forward networks, relying on learned routers whose failure modes (load imbalance, representation collapse, and training instability) are well-documented but poorly addressed from first principles.

This paper introduces **AS-MoE** (*Adaptive Structured Mixture of Experts*), an architecture that imposes a principled topological and algebraic structure on the expert ensemble. AS-MoE organizes experts along three orthogonal axes—**Generation** (g), **Flavor** (f), and **Channel** (k)—derived from a formal analogy with the quark model of particle physics, but justified independently through considerations of computational budgeting, functional specialization, and multi-view representation learning. The key insight is that the "color confinement" analogy from physics—quarks cannot be isolated, only observed in bound states—translates to a concrete architectural inductive bias: experts should not be independent, free-floating parameters, but rather *correlated components of a structured decomposition*, constrained by shared cores and mixed gradients. This analogy extends naturally to other particles of the Standard Model: the Core weights correspond to the Higgs vacuum (spontaneous symmetry breaking giving "mass" to experts), the gradient mixing matrix corresponds to gluon exchange (mediating confinement), and the generation transition logic corresponds to weak interactions via W/Z bosons (CKM-like routing transitions). Each analogy is independently justified and formally developed in the corresponding architectural section.

The contributions of this paper are:

1. A **topologically structured expert organization** (Section 4) that decouples computational budget (Generation), functional specialization (Flavor), and representational diversity (Channel) into orthogonal axes, with the g=0 generation serving as a universal shared expert analogous to DeepSeek's Shared Expert mechanism.

2. A **Core + LoRA parameter factorization** (Section 5) that separates universal, transferable core weights from view-specific, low-rank adaptation matrices, enabling efficient transfer learning without router invariance constraints.

3. A **Lagrangian budget control** mechanism (Section 7) that enforces strict FLOP constraints via dual variable optimization with exponential moving average updates, providing hard computational guarantees absent in softmax-gating approaches.

4. A **covariance gradient mixing** scheme (Section 8) that prevents expert isolation and representation collapse, with convergence guarantees formalized via Gershgorin circle analysis.

5. A **Gradient-Adaptive Router Noise** (GARN) mechanism (Section 9) that stabilizes router dynamics as a stochastic perturbation of the routing ODE, preventing mode collapse without sacrificing specialization.

6. A **Path Consensus Loss** (Section 10) that enforces cross-layer routing consistency, inspired by PathMoE, ensuring stable expert trajectories through network depth.

7. A **category-theoretic formalization** of ontogenic pre-training (Section 12) that defines the compositional bridge from attribute-level "quark" representations to language-level semantics as a structure-preserving functor.

8. A **systems design** (Section 13) grounded in Data-Oriented Design principles, including Structure-of-Arrays memory layout, fused Grouped GEMM kernels, persistent kernels via FlashDMoE, and fused computation-communication via NVSHMEM.

9. A comprehensive **training methodology** (Section 14) incorporating ontogenic pre-training, EWC++ for catastrophic forgetting prevention, and routing replay buffers.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 provides a concrete, implementation-oriented description of the AS-MoE model with diagrams and configurations. Sections 4–10 detail the core architecture components with theoretical justification. Section 11 develops the mathematical foundations. Section 12 presents the ontogenic pre-training framework. Section 13 covers systems design. Section 14 describes the full training methodology, including hyperparameters (Table 3), initialization (Table 5), component schedules (Table 4), a training pipeline diagram (Figure 5), the complete training loop (Algorithm 4), and a debugging guide (Section 14.8). Section 15 provides reference implementations. Appendix A contains the engineering PRD.

2 Related Work

2.1 Sparse Mixture of Experts

The MoE paradigm traces its origins to Jacobs et al. (1991) and Jordan and Jacobs (1994), who introduced competitive learning among expert networks gated by a learned router. The modern instantiation in transformer architectures was pioneered by Shazeer et al. (2017) with the Sparsely-Gated MoE layer, which demonstrated that top-k routing could scale model capacity with minimal per-token cost. Subsequent work—Switch Transformers (Fedus et al., 2022), GShard (Lepikhin et al., 2021), and the Suite of MoE models—established the standard pattern: a linear projection router producing softmax weights, followed by top-k selection and weighted expert computation.

A persistent challenge in MoE architectures is *load imbalance*: routers tend to collapse onto a small subset of experts, leaving others untrained. Auxiliary losses such as the load-balancing loss (Fedus et al., 2022) and the z-loss (Zoph et al., 2022) provide soft penalties but offer no hard guarantees on FLOP budgets. Expert Choice routing (Zhou et al., 2022) inverts the routing direction—experts select their top tokens rather than tokens selecting experts—achieving perfect load balance by construction. However, Expert Choice is fundamentally incompatible with autoregressive generation, where tokens arrive sequentially and cannot be preemptively ranked. AS-MoE addresses this through a *hybrid routing* strategy (Section 6): Expert Choice during pre-training for load-balanced optimization, and Token Choice during inference for causal compatibility.

2.2 Shared Experts and Structured Routing

DeepSeek-V2 (Dai et al., 2024) introduced the Shared Expert mechanism, reserving a subset of experts that process *every* token regardless of routing decisions. This architectural innovation provides a universal representational backbone that captures common knowledge, while routed experts handle specialization. AS-MoE generalizes this concept through its Generation axis: $g{=}0$ serves as the Shared Expert, but higher generations ($g{=}1, 2, \ldots$) provide progressively larger computational budgets for tokens requiring deeper processing. This hierarchical budget structure is absent in DeepSeek, where the shared/routed distinction is binary.

PathMoE (2024) introduced the concept of *path consistency* across layers, encouraging tokens to follow stable expert trajectories through network depth rather than independently routing at each layer. This reduces representational fragmentation and improves training stability. AS-MoE incorporates this insight through the Path Consensus Loss (Section 10), which aligns

routing distributions across adjacent layers while preserving the flexibility to re-route when representational demands change.

2.3 Parameter-Efficient Transfer Learning

LoRA (Hu et al., 2022) demonstrated that low-rank adaptation matrices applied to frozen pre-trained weights can achieve transfer performance comparable to full fine-tuning at a fraction of the parameter cost. The IniLoRA analysis (2024) further established that proper initialization of LoRA matrices—specifically, Kaiming initialization with appropriate scaling—is critical for stable training. AS-MoE integrates LoRA not as a post-hoc adaptation mechanism, but as a *first-class architectural component*: every expert is factored into a universal Core (shared across views and frozen during transfer) and a view-specific LoRA adapter (trainable). This design eliminates the need for router invariance during transfer, as the Core provides a stable representational anchor.

2.4 Router Stability and Noise Injection

Router collapse—where a single expert receives all tokens—is a well-known failure mode. Prior solutions include load-balancing auxiliary losses (Fedus et al., 2022), jitter noise on router logits (Lepikhin et al., 2021), and expert capacity constraints (Lewis et al., 2021). However, these are heuristic patches without formal stability guarantees. The GARN mechanism introduced in this paper (Section 9) provides a principled approach: router noise is derived from the gradient of the routing distribution, ensuring that perturbation magnitude scales with the risk of collapse. This is formalized through the lens of stochastic stabilization of dynamical systems (Arnold, 2013).

2.5 Fused Kernels and Systems Optimizations

Recent systems work has significantly improved MoE throughput. Megablocks (Gale et al., 2023) demonstrated that sparse MoE can be reformulated as block-sparse operations amenable to GPU optimization. SonicMoE and FlashDMoE (2024) introduced persistent kernels that reduce kernel launch overhead and improve memory locality for expert computation. Grouped GEMM (2024) enables efficient batched matrix multiplication across expert groups with varying dimensions. AS-MoE's systems design (Section 13) builds on these advances with a Structure-of-Arrays memory layout optimized for the Generation/Flavor/Channel topology, and fused computation-communication via NVSHMEM for distributed training.

3 Model Architecture

This section provides a concrete, implementation-oriented description of the AS-MoE model. While subsequent sections develop the theoretical justification for each component, the goal here is to answer a single question: *what exactly do we build, and how does a token pass through it?*

3.1 AS-MoE as a Drop-In Replacement for the FFN Layer

AS-MoE is not a standalone model—it is a **sparsely-activated replacement for the feed-forward network (FFN) layer** in a standard Transformer. Every other component of the Transformer (multi-head latent attention (MLA), layer normalization, residual connections, positional embeddings) remains unchanged. A Transformer with L layers has L MoE layers, each independently parameterized.

The key difference from a standard FFN is that instead of a single, fixed $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$ applied to every token, the AS-MoE layer maintains a *structured ensemble* of

$N_e = G \times F \times K$ experts, each with its own (factored) weight matrices. A routing mechanism selects which experts process each token, and only the selected experts are computed—the rest contribute zero FLOPs for that token.

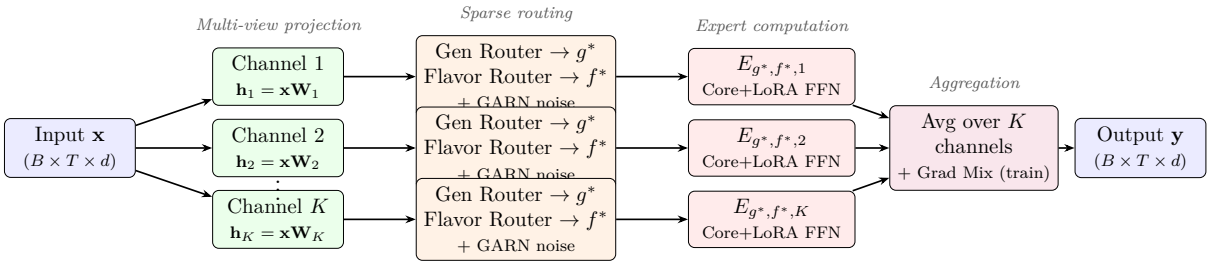

Figure 1: AS-MoE Layer: data flow for a single token. The input is projected into K channels, each independently routed and processed by a selected expert, then averaged. During training, gradient mixing (Section 8) is applied in the backward pass.

3.2 The Expert Grid: A Three-Dimensional Structure

The experts within a single AS-MoE layer are organized as a three-dimensional grid indexed by (g, f, k). Figure 2 visualizes this structure for a typical configuration $G{=}3, F{=}4, K{=}2$.

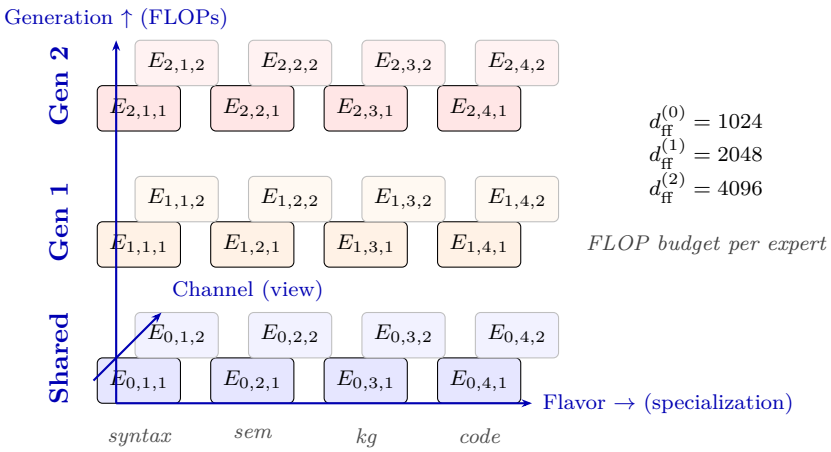

Figure 2: The AS-MoE expert grid for $G{=}3, F{=}4, K{=}2$ (24 experts per layer). Generation 0 (blue) is the Shared Expert; higher generations (orange, red) have progressively larger FFN dimensions. Each flavor specializes in a functional role. Channel 2 (lighter) is offset to show depth.

The grid structure determines the computational cost per token. A token routed to generation g^* activates all generations $\leq g^*$ (curriculum logic), and within each generation, the top-m flavors per channel are selected. For a typical setting ($g^*{=}1$, $m{=}1$, $K{=}2$), a token is processed by:

- Generation 0: $K \times m = 2$ experts (Shared Expert, always active)
- Generation 1: $K \times m = 2$ experts (if $g^* \geq 1$)
- Total active experts: 4 out of 24, yielding a sparsity ratio of $4/24 \approx 17\%$.

3.3 Inside a Single Expert: Core + LoRA

Each expert $E_{g,f,k}$ is a two-layer FFN (up-projection → activation → down-projection) with factored weights. Figure 3 shows the factorization.

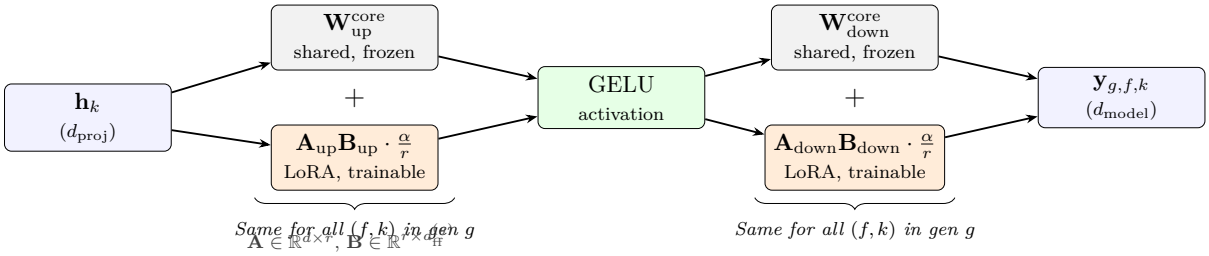

Figure 3: Core + LoRA factorization inside a single expert. The Core weights (gray) are shared across all flavors and channels within the same generation and frozen during transfer. The LoRA adapters (orange) are specific to each (g, f, k) triple and trainable. At initialization, $\mathbf{B} = \mathbf{0}$, so the expert output equals the Core output.

The parameter count per expert is dramatically reduced by the factorization. For a generation-g expert with $d = d_{\text{model}} = 4096$ and $d_{\text{ff}}^{(g)} = 2048$:

- **Full-rank**: $2 \times d \times d_{\text{ff}} = 2 \times 4096 \times 2048 = 16.8$M parameters per expert.
- **Core + LoRA** (r=8): Core (16.8M, shared) + LoRA ($2 \times (d + d_{\text{ff}}) \times r = 2 \times 6144 \times 8 = 98$K per expert).
- **Reduction**: 98K/16.8M $\approx 0.6\%$ additional parameters per expert beyond the shared Core.

3.4 The Forward Pass: Step by Step

We now trace the complete path of a single token $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ through one AS-MoE layer. This is the concrete procedure that the pseudocode in Algorithm 1 implements.

Step 1: Multi-View Projection

The token is projected into K independent representational views:

$$\mathbf{h}_k = \mathbf{x} \cdot \mathbf{W}_k, \quad \mathbf{W}_k \in \mathbb{R}^{d_{\text{model}} \times d_{\text{proj}}}, \quad k = 1, \ldots, K. \tag{1}$$

Each $\mathbf{W}_k$ is a learned projection that encourages the channel to focus on a different aspect of the input (enforced by the orthogonality loss $\mathcal{L}_{\text{ortho}}$). For K=2, $d_{\text{proj}} = d_{\text{model}}/2 = 2048$.

Step 2: Generation Routing (with GARN + STE)

Within each channel, the generation router determines the computational budget level:

$$\mathbf{z}_g^{(k)} = \mathbf{h}_k \cdot \mathbf{W}_g^{(k)} + \boldsymbol{\phi}_g^{(k)}, \quad \pi_g^{(k)} = \text{softmax}\left(\frac{\mathbf{z}_g^{(k)} + \sigma_{\text{GARN}} \odot \epsilon}{\tau_g}\right), \quad g^* = \text{argmax}_g\, \pi_g^{(k)}. \tag{2}$$

The STE trick makes the forward pass use the hard assignment g^* while the backward pass receives gradients through the soft π_g. GARN noise σ_{GARN} (Section 9) prevents router collapse. Generation routing is *shared across channels* via the consensus loss $\mathcal{L}_{\text{cons}}$.

Step 3: Curriculum Execution

The token is processed by *all* generations from 0 to g^*:

$$\mathbf{y}_k = \sum_{g=0}^{g^*} \sum_{f \in \mathcal{S}_f^{(g,k)}} \pi_f^{(g,k)} \cdot \mathrm{FFN}_{g,f,k}(\mathbf{h}_k). \tag{3}$$

This means a token routed to generation 2 receives contributions from the Shared Expert (g=0), the mid-budget experts (g=1), and the high-budget experts (g=2). The Shared Expert always contributes, providing a stable baseline representation.

Step 4: Flavor Routing (with GARN)

Within each active generation $g \leq g^*$ and channel k, the flavor router selects the top-m functional specialists:

$$\mathbf{z}_f^{(g,k)} = \mathbf{h}_k \cdot \mathbf{W}_f^{(g,k)} + \boldsymbol{\phi}_f^{(g,k)}, \quad \pi_f^{(g,k)} = \mathrm{softmax}\left(\frac{\mathbf{z}_f^{(g,k)} + \sigma_{\mathrm{GARN}} \odot \epsilon}{\tau_f}\right), \quad \mathcal{S}_f^{(g,k)} = \text{top-}m(\pi_f^{(g,k)}). \tag{4}$$

Flavor routers are *separate per generation and channel*: $\mathbf{W}_f^{(g,k)}$ is a distinct linear layer for each (g, k) pair, allowing different generations to define different functional specializations.

Step 5: Expert Computation

Each selected expert $E_{g,f,k}$ computes:

$$\mathrm{FFN}_{g,f,k}(\mathbf{h}_k) = \sigma\left(\mathbf{h}_k \cdot \mathbf{W}_{\mathrm{up}}^{(g,f,k)}\right) \cdot \mathbf{W}_{\mathrm{down}}^{(g,f,k)}, \tag{5}$$

where the weight matrices use the Core + LoRA factorization:

$$\mathbf{W}_{\mathrm{up}}^{(g,f,k)} = \underbrace{\mathbf{W}_{\mathrm{up}}^{\mathrm{core},g}}_{\text{shared}} + \underbrace{\frac{\alpha}{r}\mathbf{A}_{\mathrm{up}}^{(g,f,k)}\mathbf{B}_{\mathrm{up}}^{(g,f,k)}}_{\text{LoRA}}, \qquad \mathbf{W}_{\mathrm{down}}^{(g,f,k)} = \underbrace{\mathbf{W}_{\mathrm{down}}^{\mathrm{core},g}}_{\text{shared}} + \underbrace{\frac{\alpha}{r}\mathbf{A}_{\mathrm{down}}^{(g,f,k)}\mathbf{B}_{\mathrm{down}}^{(g,f,k)}}_{\text{LoRA}}. \tag{6}$$

The Core weights $\mathbf{W}^{\mathrm{core},g}$ are shared across all (f, k) within the same generation g.

Step 6: Channel Aggregation

The outputs of all K channels are averaged:

$$\mathbf{y} = \frac{1}{K}\sum_{k=1}^{K} \mathbf{y}_k. \tag{7}$$

During training, the backward pass applies gradient mixing (Section 8) within each channel: instead of each expert receiving only its own gradient, correlated experts share gradient information proportionally to their activation similarity.

Step 7: Auxiliary Losses

The forward pass also computes auxiliary losses that are added to the training objective:

- **Budget loss**: $\mathcal{L}_{\mathrm{budget}} = \mu \cdot (\bar{\mathcal{B}} - B_{\mathrm{target}})$, enforced by the Lagrangian controller.
- **Consensus loss**: $\mathcal{L}_{\mathrm{cons}}$, encouraging cross-channel agreement on generation routing.
- **Diversity loss**: $\mathcal{L}_{\mathrm{div}}$, preventing flavor collapse within each (g, k) pair.
- **Orthogonality loss**: $\mathcal{L}_{\mathrm{ortho}}$, encouraging channel projections to capture distinct subspaces.

3.5 Integration into a Transformer

Figure 4 shows how the AS-MoE layer replaces the FFN within a DeepSeek-V3 [8] Transformer block. The MLA attention layer, layer norms, and residual connections are identical to DeepSeek-V3. Unlike standard Multi-Head Attention, MLA compresses the key-value cache into a low-rank latent representation, reducing memory footprint during inference while preserving attention quality.

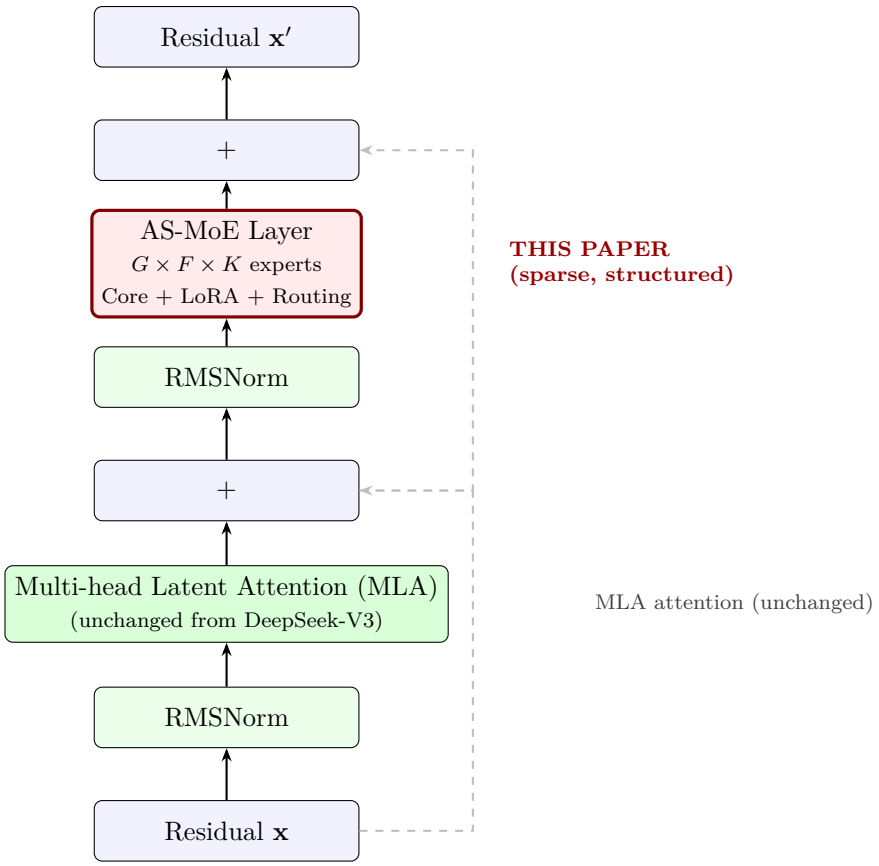

Figure 4: Transformer block with AS-MoE. The AS-MoE layer (red) replaces the standard FFN. All other components (MLA attention, RMSNorm, residual connections) are identical to DeepSeek-V3.

3.6 Concrete Model Configurations

Table 1 provides concrete specifications for three AS-MoE model sizes, following the naming convention of popular open-source models.

The "active params/token" metric is the key number for comparing AS-MoE against dense baselines: AS-MoE-S with 0.5B active parameters should be compared against a 0.5B dense Transformer, not a 1.3B one. The total parameter count (1.3B) includes all expert weights, most of which are inactive for any given token but contribute to the model's capacity for handling diverse inputs.

Table 1: Concrete AS-MoE model configurations. "Active params" denotes the number of parameters used per token at the target budget B_{target}. "Sparsity" is the fraction of experts that are inactive per token.

Hyperparameter	**AS-MoE-S**	**AS-MoE-M**	**AS-MoE-L**
Transformer backbone			
d_{model}	2048	4096	5120
n_{heads}	16	32	40
MLA latent dim d_{lat}	512	512	768
n_{layers}	16	32	40
Vocabulary size	32000	32000	32000
AS-MoE expert topology			
Generations G	2	3	3
Flavors F	4	4	8
Channels K	2	2	4
Total experts/layer	16	24	96
Expert dimensions			
d_{base}	512	1024	1024
$d_{\text{ff}}^{(0)}$ (Shared)	512	1024	1024
$d_{\text{ff}}^{(1)}$	1024	2048	2048
$d_{\text{ff}}^{(2)}$	—	4096	4096
d_{proj}	1024	2048	2560
LoRA configuration			
LoRA rank r	4	8	8
LoRA α	16	16	16
LoRA params/expert	25K	98K	123K
Routing configuration			
Top-m flavors	1	1	2
Target budget B_{target}	1.5	2.0	3.0
GARN σ_{base}	0.1	0.1	0.1
GARN η	2.0	2.0	2.0
Mix strength γ	0.1	0.1	0.05
Parameter counts			
Total params	1.3B	6.7B	13B
Active params/token	0.5B	2.1B	4.8B
Sparsity	75%	83%	63%

3.7 Training FLOP Budget Analysis

A key advantage of AS-MoE is that the per-token FLOP cost is *deterministic and controllable* via the Lagrangian budget controller. For AS-MoE-M with $B_{\text{target}} = 2.0$ and G=3:

$$\text{FLOPs per token per layer} = \underbrace{2 \cdot d \cdot d_{\text{ff}}^{(0)}}_{\text{Shared Expert (always active)}} + \underbrace{P(g^* \geq 1) \cdot 2 \cdot d \cdot d_{\text{ff}}^{(1)}}_{\text{Gen 1 (conditional)}} + \underbrace{P(g^* \geq 2) \cdot 2 \cdot d \cdot d_{\text{ff}}^{(2)}}_{\text{Gen 2 (conditional)}}, \tag{8}$$

where $P(g^* \geq g)$ is the fraction of tokens routed to generation g or higher, controlled by the Lagrangian dual variable μ. For $B_{\text{target}} = 2.0$ and budgets $[1.0, 2.0, 4.0]$, approximately 70% of tokens are routed to g^*=0, 25% to g^*=1, and 5% to g^*=2, yielding an effective FLOP multiplier of $\approx 1.35\times$ relative to a single-generation model at $d_{\text{ff}}^{(0)}$.

This is strictly less than a dense FFN with $d_{\text{ff}} = 4096$ (which costs 4× the Shared Expert), while providing the full capacity of the $d_{\text{ff}}^{(2)} = 4096$ experts for the tokens that need them. The Lagrangian controller ensures this budget is maintained as a hard constraint, not a soft penalty.

4 Expert Topology

4.1 The Three Axes of Structure

The AS-MoE expert ensemble is indexed by three orthogonal coordinates:

Definition 4.1 (Expert Index)**.** An expert $E_{g,f,k}$ is identified by the triple (g, f, k) where:

- $g \in \{0, 1, \ldots, G{-}1\}$ is the **Generation**, encoding the computational budget level. Generation g=0 is the *Shared Expert*, active for all tokens. Higher generations allocate progressively more FLOPs.
- $f \in \{1, 2, \ldots, F\}$ is the **Flavor**, encoding functional specialization (e.g., syntax, semantics, world knowledge).
- $k \in \{1, 2, \ldots, K\}$ is the **Channel** (or View), encoding representational diversity via independent projection heads.

The total number of experts is $N_e = G \times F \times K$. A typical configuration is G=3, F=4, K=2, yielding N_e=24 experts per layer, of which only a sparse subset is activated per token.

4.2 Generation: Hierarchical Budget

The Generation axis implements a hierarchical computational budget. Each generation g is associated with an intermediate dimension $d_{\text{ff}}^{(g)}$ and a FLOP budget $B^{(g)}$:

$$d_{\text{ff}}^{(g)} = d_{\text{base}} \cdot 2^g, \quad g = 0, 1, \ldots, G{-}1, \tag{9}$$

$$B^{(g)} \propto d_{\text{ff}}^{(g)} \cdot d_{\text{model}}, \tag{10}$$

where d_{base} is the minimum FFN dimension (e.g., 1024) and d_{model} is the model hidden dimension. For G=3:

$$d_{\text{ff}}^{(0)} = 1024, \quad d_{\text{ff}}^{(1)} = 2048, \quad d_{\text{ff}}^{(2)} = 4096. \tag{11}$$

The *Shared Expert* (g=0) processes every token with minimal computation, providing universal representational coverage. Tokens requiring deeper processing are routed to higher generations by the generation router. This design ensures that simple tokens (common patterns, high-frequency words) consume minimal FLOPs, while complex tokens (rare words, mathematical expressions, code) receive proportionally more compute.

Remark 4.1. The Generation hierarchy is analogous to the energy level structure in quantum mechanics: low-energy states (simple tokens) occupy ground-level generations, while high-energy excitations (complex tokens) are promoted to higher generations. The Lagrangian budget controller (Section 7) acts as the "thermostat" regulating the distribution of tokens across energy levels.

4.3 Flavor: Functional Specialization

The Flavor axis encodes functional specialization. Unlike Generation, which is determined by computational demand, Flavor is determined by the *type* of processing a token requires. The flavor router produces a distribution over F flavors, and the top-m flavors are selected per token (typically m=1 or m=2).

The critical design choice is that Flavor routing is *independent per channel*: each of the K channels computes its own flavor distribution. This allows the same token to receive different functional treatments in different representational subspaces, which is essential for handling tokens with polysemous or multifaceted semantics.

4.4 Channel: Multi-View Representation

The Channel axis implements multi-view representation learning. Each channel k applies an independent linear projection $\mathbf{W}_k \in \mathbb{R}^{d_{\text{model}} \times d_{\text{proj}}}$ to the input, producing a view-specific representation:

$$\mathbf{h}_k = \mathbf{x}\mathbf{W}_k, \quad k = 1, \ldots, K. \tag{12}$$

Routing and expert computation proceed independently within each channel, and the final output is the average across channels:

$$\text{Output} = \frac{1}{K} \sum_{k=1}^{K} \text{Channel}_k(\mathbf{h}_k). \tag{13}$$

An orthogonal projection loss $\mathcal{L}_{\text{ortho}}$ encourages channels to capture distinct representational subspaces:

$$\mathcal{L}_{\text{ortho}} = \sum_{k \neq k'} \left\| \mathbf{W}_k^\top \mathbf{W}_{k'} \right\|_F^2. \tag{14}$$

4.5 Topological Invariants

The three-axis structure gives rise to topological invariants that constrain the expert ensemble:

Proposition 4.1 (Budget Monotonicity)**.** For any two generations $g < g'$, the per-expert FLOP budget satisfies $B^{(g)} \leq B^{(g')}$, and the set of tokens routed to g' is a subset of those routed to g under the Lagrangian controller.

This follows from the curriculum logic (Section 6): a token must "pass through" lower generations before being promoted, ensuring that the Shared Expert always contributes to the representation.

Proposition 4.2 (Flavor Coverage)**.** With F flavors and top-m selection, every token is processed by at least m functional specialists, and the expected number of active flavors per batch is $\min(F, m \cdot B/|\text{batch}|)$ under load balance.

5 Parameter Factorization: Core + LoRA

5.1 Factorization Structure

Each expert $E_{g,f,k}$ factorizes its weight matrices into a **Core** (universal, shared across views) and a **LoRA adapter** (view-specific):

Definition 5.1 (Core + LoRA Factorization)**.** For expert $E_{g,f,k}$ with FFN weight matrices $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}^{(g)}}$ and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}}^{(g)} \times d_{\text{model}}}$:

$$\mathbf{W}_{\text{up}} = \underbrace{\mathbf{W}_{\text{up}}^{\text{core}}}_{\text{shared, frozen}} + \underbrace{\mathbf{A}_{\text{up}}^{(g,f,k)} \mathbf{B}_{\text{up}}^{(g,f,k)}}_{\text{LoRA, trainable}}, \tag{15}$$

$$\mathbf{W}_{\text{down}} = \underbrace{\mathbf{W}_{\text{down}}^{\text{core}}}_{\text{shared, frozen}} + \underbrace{\mathbf{A}_{\text{down}}^{(g,f,k)} \mathbf{B}_{\text{down}}^{(g,f,k)}}_{\text{LoRA, trainable}}, \tag{16}$$

where $\mathbf{A} \in \mathbb{R}^{d \times r}$, $\mathbf{B} \in \mathbb{R}^{r \times d'}$, and $r \ll d$ is the LoRA rank (typically r=4 or r=8).

The Core weights are shared across all (f, k) within the same generation g and frozen during transfer learning. The LoRA adapters are specific to each (g, f, k) triple and carry the view- and specialization-specific adaptations.

5.2 Initialization: Kaiming with Scaling

Following the IniLoRA analysis, proper initialization of LoRA matrices is critical for training stability. We adopt:

1. $\mathbf{A}$ is initialized with Kaiming uniform: $\mathbf{A}_{ij} \sim \mathcal{U}\left(-\sqrt{3/d}, \sqrt{3/d}\right)$.
2. $\mathbf{B}$ is initialized to zero: $\mathbf{B} = \mathbf{0}$.
3. A scaling factor α/r is applied to the LoRA output, where α is a hyperparameter (typically α=16).

The zero initialization of $\mathbf{B}$ ensures that the expert output is purely determined by the Core at initialization, with LoRA contributions growing smoothly from zero. The Kaiming initialization of $\mathbf{A}$ preserves the variance of activations through the low-rank projection, preventing signal attenuation or explosion in the early stages of training.

5.3 Kato Perturbation Theory Interpretation

The Core + LoRA factorization admits a natural interpretation through Kato's perturbation theory for linear operators. Consider the Core weight $\mathbf{W}^{\text{core}}$ as the *unperturbed operator* and the LoRA term $\mathbf{AB}$ as the *perturbation*:

$$\mathbf{W} = \mathbf{W}^{\text{core}} + \epsilon \cdot \Delta\mathbf{W}, \quad \Delta\mathbf{W} = \mathbf{AB}, \quad \epsilon = \frac{\alpha}{r}. \tag{17}$$

Kato's theory provides bounds on how the spectral properties of $\mathbf{W}$ deviate from $\mathbf{W}^{\text{core}}$ under perturbation. Specifically, if λ_i are the eigenvalues of $\mathbf{W}^{\text{core}}$, then the perturbed eigenvalues λ_i' satisfy:

$$|\lambda_i' - \lambda_i| \leq \|\Delta\mathbf{W}\|_2 \cdot \frac{\alpha}{r} + O\left(\left(\frac{\alpha}{r}\right)^2\right). \tag{18}$$

This guarantees that the LoRA adaptation does not drastically alter the spectral structure of the Core, which is critical for maintaining stable representations during transfer. The low-rank structure of the perturbation ($\text{rank}(\Delta\mathbf{W}) = r \ll d$) further constrains the perturbation to a low-dimensional subspace, limiting its effect on the operator's dominant eigenvectors.

Remark 5.1 (Higgs Vacuum Analogy)**.** The Core + LoRA factorization admits a second, complementary interpretation through the Higgs mechanism of particle physics. In the Standard Model, the Higgs field permeates all of space with a nonzero vacuum expectation value $\langle\phi\rangle \neq 0$. This spontaneous symmetry breaking gives mass to particles: without the Higgs field, all particles would be massless and indistinguishable; with it, each particle acquires a specific mass proportional to its coupling to the Higgs field. The Higgs vacuum is *not* empty—it is a structured, nonzero background that breaks the symmetry of the theory.

In AS-MoE, the Core weights $\mathbf{W}^{\text{core}}$ play the role of the Higgs vacuum. At initialization, $\mathbf{B} = \mathbf{0}$, so all experts within the same generation produce identical outputs—a state of perfect symmetry where no specialization exists. The nonzero Core breaks this symmetry: every expert begins with a shared, nontrivial representational foundation (the "Higgs vacuum"), and the LoRA adapters $\mathbf{AB}$ are the "excitations" around this vacuum that give each expert its individual identity. The scaling α/r controls the coupling strength to the vacuum, exactly as the Yukawa coupling g_Y controls the mass $m = g_Y\langle\phi\rangle$ in the Higgs mechanism.

The "mass" of an expert—its FLOP budget $B^{(g)} = d_{\text{ff}}^{(g)} \cdot d_{\text{model}}$—is thus a consequence of the generation structure, which is itself a form of symmetry breaking: without the generation hierarchy (all experts at the same d_{ff}), there would be no distinction between "light" and "heavy" experts, just as without the Higgs mechanism all fermions would be massless. The Kato perturbation bound (Section 5) then acquires a physical interpretation: it guarantees that the LoRA "excitations" do not destabilize the Higgs vacuum, preserving the mass hierarchy established by the Core.

5.4 Spontaneous Symmetry Breaking in Core-LoRA Dynamics

The Higgs vacuum analogy (Remark 5.1) can be formalized by modeling the local loss landscape around the initialization state. At initialization, $\mathbf{B} = \mathbf{0}$, and the active ensemble possesses an S_N permutation symmetry (all experts within a generation produce identical outputs). The parameter update for the LoRA adapter $\Delta\mathbf{W} = \mathbf{AB}$ is governed by the risk functional $\mathcal{L}$.

By performing a Taylor expansion of the loss landscape with an added diversity regularization, the effective potential $\mathcal{V}(\mathbf{B})$ governing the adapter's dynamics takes the form of a Ginzburg-Landau potential:

$$\mathcal{V}(\mathbf{B}) \approx -\frac{\mu_{\text{div}}^2}{2}\|\mathbf{B}\|_F^2 + \frac{\lambda}{4}\|\mathbf{B}\|_F^4 + \frac{1}{2}\operatorname{tr}(\mathbf{B}^\top\mathbf{H}_{\text{task}}\mathbf{B}), \tag{19}$$

where:

- The critical "mass" term $-\frac{\mu_{\text{div}}^2}{2}\|\mathbf{B}\|_F^2$ is induced by the routing diversity loss $\mathcal{L}_{\text{div}}$. By actively penalizing redundant routing, it repels experts from producing identical outputs, thereby destabilizing the origin ($\mathbf{B} = \mathbf{0}$).
- The quartic term $\frac{\lambda}{4}\|\mathbf{B}\|_F^4$ represents the natural capacity constraint and weight decay, bounded by Kato's perturbation limits (Proposition 11.2).
- $\mathbf{H}_{\text{task}}$ is the task-specific Hessian projected onto the LoRA subspace.

When the repulsive force μ_{div}^2 exceeds the lowest eigenvalue of $\mathbf{H}_{\text{task}}$, the symmetric minimum at $\mathbf{B} = \mathbf{0}$ becomes a local maximum. The system undergoes spontaneous symmetry breaking, rolling down into one of the specialized global minima where $\|\mathbf{B}\|_F > 0$. The Core weight $\mathbf{W}_g^{\text{core}}$ establishes the symmetric vacuum, and the LoRA rank r defines the continuous degrees of freedom (analogous to Goldstone bosons) through which the symmetry is dynamically broken during training.

Remark 5.2 (Physical Interpretation of the Ginzburg-Landau Formalism)**.** The Ginzburg-Landau potential provides a rigorous mathematical framework for understanding why diversity-encouraging losses are not merely heuristic regularization, but are essential for *phase transition* in the parameter space. Without $\mathcal{L}_{\text{div}}$, the system remains trapped in the symmetric phase ($\mathbf{B} = \mathbf{0}$) and experts never specialize—analogous to a universe without spontaneous symmetry breaking, where all particles remain massless and indistinguishable. The strength of the diversity loss μ_{div}^2 determines whether the phase transition occurs at all (first-order vs. second-order), and the LoRA rank r controls the dimensionality of the order parameter manifold.

5.5 Transfer Learning Protocol

During transfer to a new domain or task:

1. The Core weights $\mathbf{W}^{\text{core}}$ are **frozen**.
2. All LoRA adapters are **re-initialized** (or selectively reset).
3. The generation and flavor routers are **re-initialized**, allowing the new domain to develop its own routing topology.
4. Channel projections $\mathbf{W}_k$ may be **fine-tuned** with a reduced learning rate.

This protocol eliminates the need for *router invariance*—the constraint that the same tokens must be routed to the same experts across domains—because the Core provides a domain-agnostic representational foundation. The LoRA adapters learn domain-specific adaptations *within* the subspace defined by the Core, and the routers are free to discover new specialization patterns.

6 Multi-Channel Routing

6.1 Generation Routing: Consensus

The generation router determines the computational budget level for each token. Unlike flavor routing, which can differ across channels, generation routing enforces *cross-channel consensus*: all channels agree on the generation level for a given token.

Definition 6.1 (Generation Routing)**.** For input $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, the generation logits are computed as:

$$\mathbf{z}_g = \mathbf{x}\mathbf{W}_g + \boldsymbol{\phi}_g, \quad \mathbf{W}_g \in \mathbb{R}^{d_{\text{model}} \times G}, \quad \boldsymbol{\phi}_g \in \mathbb{R}^G, \tag{20}$$

where $\boldsymbol{\phi}_g$ is a learnable bias. The generation distribution is:

$$\pi_g = \text{softmax}(\mathbf{z}_g / \tau_g), \tag{21}$$

with temperature $\tau_g > 0$. The generation assignment is:

$$g^* = \text{argmax}_g \, \pi_g. \tag{22}$$

During training, a Straight-Through Estimator (STE) is used: the forward pass uses the hard assignment g^*, while the backward pass flows through the soft distribution π_g. This ensures differentiability while maintaining sparse activation.

Curriculum Logic. A key inductive bias is that tokens must "graduate" through generations. A token at generation g receives contributions from *all* generations $\leq g$:

$$\text{Output}_{\text{gen}} = \sum_{g'=0}^{g^*} \text{Expert}_{g'}(\mathbf{x}). \tag{23}$$

This curriculum ensures that the Shared Expert (g=0) always contributes, providing a stable representational baseline, while higher generations add incremental specialization.

Remark 6.1 (Weak Interaction Analogy: W/Z Bosons and the CKM Matrix)**.** The generation routing and curriculum logic admit an analogy with the weak interaction of the Standard Model. In particle physics, the weak force is mediated by the $W^{\pm}$ and Z^0 bosons. The charged-current interaction ($W^{\pm}$) is the only fundamental interaction that *changes quark flavor across generations*: $d \to u + W^-$, $s \to c + W^-$, etc. The neutral-current interaction (Z^0) mediates same-generation processes without flavor change.

In AS-MoE, the curriculum logic implements a "charged-current" transition: a token promoted from generation g to g+1 undergoes a "flavor change" in the budget dimension, analogous to a quark being promoted to a higher generation via W-boson emission. The flavor routing within a fixed generation (Section 6) corresponds to the "neutral current" (Z^0): the token's functional specialization changes without altering its budget level.

The probabilities of inter-generation transitions are governed by the Cabibbo–Kobayashi–Maskawa (CKM) matrix V_{CKM} in the Standard Model, which quantifies the mixing between quark mass eigenstates and weak eigenstates. By analogy, we define the **routing transition matrix** $\mathbf{V} \in \mathbb{R}^{G\times G}$ for AS-MoE:

$$V_{gg'} = P(\text{token at gen } g \text{ transitions to gen } g'), \tag{24}$$

where the transition probabilities are determined by the generation router (Definition 6.1). Unlike the CKM matrix, $\mathbf{V}$ is not unitary: the curriculum constraint imposes that $V_{gg'} = 0$ for $g' < g$ (no downward transitions), and the diagonal dominance of $\mathbf{V}$ reflects the fact that most tokens remain at their current generation. The off-diagonal entries $V_{g,g+1}$ measure the rate of "generation promotion," analogous to the CKM elements $|V_{us}|$, $|V_{cb}|$, $|V_{tb}|$ that quantify cross-generation mixing. The Lagrangian budget controller (Section 7) regulates these transition rates: increasing μ suppresses upward transitions (reducing off-diagonal entries), while decreasing μ allows more promotions.

This analogy provides a principled framework for analyzing the routing dynamics: the transition matrix $\mathbf{V}$ can be estimated from training statistics and its deviation from diagonal structure measures the "weak interaction strength" of the generation router.

6.2 Flavor Routing: Diversification

Within each channel k and generation g, the flavor router determines which functional specialists process the token:

Definition 6.2 (Flavor Routing)**.** For the projected representation $\mathbf{h}_k = \mathbf{x}\mathbf{W}_k$ within channel k and generation g:

$$\mathbf{z}_f^{(g,k)} = \mathbf{h}_k \mathbf{W}_f^{(g,k)} + \boldsymbol{\phi}_f^{(g,k)}, \tag{25}$$

$$\pi_f^{(g,k)} = \text{softmax}(\mathbf{z}_f^{(g,k)}/\tau_f), \tag{26}$$

$$\mathcal{S}_f^{(g,k)} = \text{top-}m\left(\pi_f^{(g,k)}\right), \tag{27}$$

where $\mathcal{S}_f^{(g,k)}$ is the set of m selected flavors. The flavor weights are renormalized over the selected set. Critically, the flavor router is conditioned on both channel k and generation g: different generations may assign different functional specializations to the same token, reflecting the fact that computational depth interacts with functional type.

A **diversity loss** encourages balanced flavor utilization within each generation-channel pair:

$$\mathcal{L}_{\text{div}} = \frac{1}{G\cdot K}\sum_{g=1}^{G}\sum_{k=1}^{K}\left[\frac{1}{F}\sum_{f=1}^{F}\left(\bar{\pi}_f^{(g,k)} - \frac{1}{F}\right)^2\right], \tag{28}$$

where $\bar{\pi}_f^{(g,k)}$ is the average routing probability for flavor f across the batch, computed independently for each generation g and channel k. Averaging per (g, k) prevents mixing different routing distributions across generations, which would obscure imbalances specific to a particular budget level.

6.3 Hybrid Routing: Expert Choice + Token Choice

A critical practical consideration is the incompatibility of Expert Choice (EC) routing with autoregressive generation. EC routing allows experts to select their top tokens, achieving perfect load balance by construction. However, EC requires access to the full token sequence before routing, which is impossible during causal inference where tokens arrive one at a time.

AS-MoE resolves this with a **Hybrid Routing** strategy:

1. **Pre-training phase**: Expert Choice routing is used for both generation and flavor assignment. This maximizes training efficiency and load balance.
2. **Fine-tuning / Inference phase**: Token Choice routing (standard top-k) is used, with the routers initialized from the EC-trained weights. The transition from EC to TC is smoothed by a brief fine-tuning period with TC routing and an auxiliary load-balancing loss.

This hybrid approach leverages the training efficiency of EC while ensuring causal compatibility at inference time.

7 Budget Control via Lagrangian Duality

7.1 Problem Formulation

The goal is to enforce a strict FLOP budget $\mathcal{B}$ per token, where:

$$\mathcal{B} = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\sum_{g,f,k} \mathbb{1}[E_{g,f,k} \text{ active}] \cdot B^{(g)}\right]. \tag{29}$$

Standard load-balancing losses provide soft penalties but no hard guarantees. We formulate budget control as a constrained optimization problem:

$$\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{task}}(\boldsymbol{\theta}) \quad \text{s.t.} \quad \mathcal{B}(\boldsymbol{\theta}) \leq B_{\text{target}}. \tag{30}$$

7.2 Lagrangian Dual Variable with Two-Phase Update

Introducing a Lagrangian multiplier $\mu \geq 0$, the Lagrangian is:

$$\mathcal{L}_{\text{lagr}}(\boldsymbol{\theta}, \mu) = \mathcal{L}_{\text{task}}(\boldsymbol{\theta}) + \mu\left(\mathcal{B}(\boldsymbol{\theta}) - B_{\text{target}}\right). \tag{31}$$

The dual variable μ is updated via a two-phase procedure that separates budget estimation from multiplier optimization:

Phase A: EMA for budget estimation. The empirical budget is tracked via EMA to reduce variance:

$$\bar{\mathcal{B}}^{(t)} = \beta \cdot \bar{\mathcal{B}}^{(t-1)} + (1-\beta) \cdot \mathcal{B}_{\text{batch}}^{(t)}, \tag{32}$$

where $\beta \in [0.9, 0.999]$ is the EMA decay rate and $\mathcal{B}_{\text{batch}}^{(t)}$ is the empirical FLOP budget computed from the actual (hard) routing decisions at step t.

Phase B: Gradient step for μ. The multiplier is updated via gradient ascent on the dual objective:

$$\mu^{(t+1)} = \max\left(0,\ \mu^{(t)} + \eta_\mu \left(\bar{\mathcal{B}}^{(t)} - B_{\text{target}}\right)\right), \tag{33}$$

where $\eta_\mu > 0$ is the dual learning rate. This separation ensures that μ responds to the smoothed budget estimate (not noisy per-batch fluctuations) while performing true gradient-based optimization of the Lagrangian.

Theorem 7.1 (Budget Convergence)**.** Under the two-phase update with EMA decay $\beta \in (0, 1)$ and dual learning rate η_μ satisfying $\sum_t \eta_\mu = \infty$, $\sum_t \eta_\mu^2 < \infty$ (Robbins-Monro conditions), if the empirical budget $\mathcal{B}^{(t)}$ is bounded, then $\mu^{(t)}$ converges almost surely to the optimal dual variable μ^*, and the expected budget converges:

$$\lim_{t\to\infty} \mathbb{E}\left[\bar{\mathcal{B}}^{(t)}\right] \leq B_{\text{target}} + O\left(\eta_\mu\right). \tag{34}$$

The budget loss is added to the total training objective:

$$\mathcal{L}_{\text{budget}} = \mu \cdot \left(\mathcal{B} - B_{\text{target}}\right). \tag{35}$$

When the budget is exceeded, μ increases, penalizing high-generation routing and pushing tokens toward lower (cheaper) generations. When the budget is under-utilized, μ decreases toward zero, relaxing the constraint.

8 Covariance Gradient Mixing

8.1 The Expert Isolation Problem

In standard MoE training, gradients flow only through the active experts for each token. Over time, this leads to *expert isolation*: experts converge to independent local optima, losing the ability to cooperate. In extreme cases, experts develop redundant representations (mode collapse), wasting capacity.

8.2 Gradient Mixing Mechanism

AS-MoE addresses expert isolation through **covariance gradient mixing**. The key idea is to mix the gradients of different experts proportionally to their activation correlation, ensuring that correlated experts receive similar gradient signals.

Definition 8.1 (Gradient Mixing)**.** Let $\{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N\}$ be the outputs of the $N = G \times F$ active experts *within a single channel* k, with routing weights $\{w_1, w_2, \ldots, w_N\}$. The mixed output is:

$$\mathbf{y} = \sum_{i=1}^{N} w_i \mathbf{y}_i. \tag{36}$$

In the backward pass, the gradient mixing matrix $\mathbf{M}$ is computed from the activation covariance:

$$\mathbf{R}_{ij} = \frac{\langle \mathbf{y}_i, \mathbf{y}_j \rangle}{\|\mathbf{y}_i\| \cdot \|\mathbf{y}_j\|}, \quad \mathbf{M} = \mathbf{I} + \gamma \cdot \mathbf{R}, \tag{37}$$

where $\gamma > 0$ is the mixing strength. The mixed gradients are:

$$\nabla_{\mathbf{y}_i}^{\text{mixed}} = \sum_j \mathbf{M}_{ji} \cdot \nabla_{\mathbf{y}_j}^{\text{raw}}. \tag{38}$$

Remark 8.1. Gradient mixing is applied **independently within each channel**, not globally across all $G \times F \times K$ experts of the layer. This design choice is motivated by two considerations: (1) experts within the same channel operate on the same projected representation $\mathbf{h}_k = \mathbf{x}\mathbf{W}_k$ and are thus directly comparable, whereas experts across channels operate in different representational subspaces; (2) per-channel mixing incurs an $O((GF)^2)$ correlation computation per channel rather than $O((GFK)^2)$ for global mixing, a significant reduction for $K \geq 2$. The N in Definition 8.1 refers to $N = G \times F$ (experts per channel), not $N_e = G \times F \times K$ (total experts per layer). In distributed settings, a DDP all-gather may extend mixing to cross-GPU expert correlations within the same channel.

The mixing matrix $\mathbf{M}$ ensures that:

- Each expert receives its own gradient (the $\mathbf{I}$ term).
- Additionally, each expert receives a fraction γ of the gradients from correlated experts (the $\gamma\mathbf{R}$ term).
- Strongly correlated experts receive similar gradient signals, preventing drift.

Remark 8.2 (Gluon Exchange Analogy)**.** The structure of the mixing matrix $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$ admits a precise analogy with gluon exchange in Quantum Chromodynamics (QCD). In QCD, quarks interact by exchanging gluons—the gauge bosons of the strong force—which mediate the binding that prevents quark isolation (confinement). In AS-MoE, the diagonal term $\mathbf{I}$ corresponds to "free propagation" of gradients (each expert receives only its own gradient signal), while the off-diagonal term $\gamma\mathbf{R}$ corresponds to "gluon exchange": the correlation matrix $\mathbf{R}$ mediates the transfer of gradient information between correlated experts, binding them together and preventing expert isolation. Just as gluon exchange is responsible for confinement in QCD, the $\gamma\mathbf{R}$ term is responsible for *gradient confinement* in AS-MoE—the property that no expert can diverge arbitrarily from its correlated neighbors. The mixing strength γ plays the role of the strong coupling constant α_s: too small γ yields free (isolated) experts, while too large γ (exceeding the Gershgorin bound $1/(N-1)$) causes "asymptotic freedom" to fail and gradient directions to reverse. The eight gluon types in QCD (the $SU(3)$ color octet) correspond to the $N(N-1)/2$ independent off-diagonal entries of $\mathbf{R}$, each mediating a pairwise expert interaction.

8.3 Gauge-Theoretic Perspective on Gradient Confinement

The covariance gradient mixing (Section 8) extends the particle physics analogy to Quantum Chromodynamics (QCD). We formalize this by treating the expert activations within a channel as a vector-valued multiplet $\Psi = (\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N)^\top \in \mathbb{R}^{N \times d}$.

In standard MoE, the gradient update for each expert is completely decoupled. This configuration possesses a trivial, local gauge symmetry where each expert optimizes independently, leading to the *expert isolation* failure mode.

The mixing matrix $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$ acts as a discrete connection (gauge field) on the expert graph. The modified gradient descent step substitutes the standard derivative with a covariant derivative $\mathcal{D}$:

$$\mathcal{D}\mathbf{y}_i = \nabla_{\mathbf{y}_i}\mathcal{L}_{\text{raw}} + \gamma \sum_{j \neq i} R_{ij} \nabla_{\mathbf{y}_j}\mathcal{L}_{\text{raw}}. \tag{39}$$

In this formulation, the correlation coefficients R_{ij} act as the connection coefficients (analogous to the gluon field tensor $G^a_{\mu\nu}$ in QCD). The parameter γ acts as the strong coupling constant α_s:

1. **Asymptotic Freedom:** When experts are functionally orthogonal ($R_{ij} \to 0$), the interaction term vanishes. Experts behave as non-interacting, "free" parameters and optimize locally.

2. **Confinement:** When experts become highly correlated across redundant modes ($R_{ij} \to 1$), the interaction term scales up. The strongly coupled gradients force the experts to update cohesively, preventing them from splitting into disconnected sub-clusters.

Thus, gradient mixing preserves the structural integrity of the expert multiplet, ensuring that functional specialization (Flavor) defines the routing, rather than arbitrary local minima trapping isolated experts. The condition number analysis (Theorem 8.2) further shows that the "coupling constant" γ must remain below the Gershgorin bound to avoid the non-perturbative regime where the gauge-theoretic description breaks down, precisely mirroring the behavior of α_s in QCD.

8.4 Convergence Guarantee

Lemma 8.1 (Gershgorin Stability)**.** The eigenvalues λ_i of the mixing matrix $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$ satisfy:

$$\lambda_i \in \left[1 - \gamma \sum_{j \neq i} |R_{ij}|,\ 1 + \gamma \sum_{j \neq i} |R_{ij}|\right]. \tag{40}$$

Since $|R_{ij}| \leq 1$ (normalized correlation), the eigenvalues are positive for $\gamma < 1/(N-1)$, ensuring that gradient mixing does not reverse gradient directions and preserves convergence of SGD.

This guarantee is critical: without it, aggressive mixing could destabilize training by introducing gradient directions opposite to the true gradient. The Gershgorin bound provides a safe operating range for γ.

Theorem 8.2 (Conditioning of Gradient Mixing)**.** Let $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$ be the gradient mixing matrix. While Lemma 8.1 ensures positive eigenvalues for $\gamma < 1/(N-1)$, the optimization convergence rate depends critically on the condition number $\kappa(\mathbf{M}) = \lambda_{\max}(\mathbf{M})/\lambda_{\min}(\mathbf{M})$. Using the Gershgorin circle bounds, the condition number is strictly bounded by:

$$\kappa(\mathbf{M}) \leq \frac{1 + \gamma \max_i \sum_{j \neq i} |R_{ij}|}{1 - \gamma \max_i \sum_{j \neq i} |R_{ij}|} \leq \frac{1 + \gamma(N-1)}{1 - \gamma(N-1)}. \tag{41}$$

Remark 8.3 (The Asymptotic Freedom Trade-off)**.** Theorem 8.2 exposes the mathematical trade-off underlying the "gluon exchange" analogy (Remark 8.2). As $\gamma \to 1/(N-1)$, the binding force between experts approaches maximum strength (preventing isolation entirely), but the condition number $\kappa(\mathbf{M}) \to \infty$. In optimization theory, a high condition number in the gradient pre-conditioner leads to severe "zig-zagging" oscillations and exponentially throttles the SGD convergence rate. Therefore, the optimal mixing strength γ^* must balance the necessity of confinement (avoiding mode collapse) against this conditioning penalty, placing the optimal regime strictly bounded away from the limit, roughly at $\gamma \approx \frac{1}{2(N-1)}$.

This trade-off has a direct physical counterpart: in QCD, the strong coupling constant α_s grows at low energies (confinement), making perturbative calculations unreliable. Similarly, in AS-MoE, large γ ensures gradient confinement but degrades the condition number, making the optimization landscape ill-conditioned. The "asymptotic freedom" regime ($\gamma \ll 1/(N-1)$, where experts interact weakly and optimize efficiently) corresponds to the high-energy regime of QCD where quarks behave as free particles.

8.5 Memory-Efficient Implementation

To prevent memory explosion from storing all expert outputs, the implementation saves activations in `bfloat16` and computes the correlation matrix $\mathbf{R}$ in reduced precision. The dimension-preserving backward pass is detailed in Section 15. In distributed settings (DDP), an all-gather operation computes the global correlation matrix across GPUs, ensuring that mixing reflects the true cross-expert correlation structure rather than a per-GPU approximation.

9 Router Stability: GARN

9.1 Router Collapse as Dynamical System

The routing process can be modeled as a discrete-time dynamical system. Let $\pi^{(t)}$ denote the routing distribution at training step t. The router update can be written as:

$$\pi^{(t+1)} = F\left(\pi^{(t)}, \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(t)}\right), \tag{42}$$

where F is the update mapping determined by the optimizer and loss landscape.

Router collapse occurs when F has an attracting fixed point at a degenerate distribution $\pi^* = \mathbf{e}_i$ (all mass on a single expert i). This happens when:

1. Expert i receives more tokens $\rightarrow$ more gradient updates $\rightarrow$ better performance $\rightarrow$ even more tokens (positive feedback loop).
2. Under-trained experts cannot compete $\rightarrow$ receive no tokens $\rightarrow$ never improve (starvation).

9.2 Gradient-Adaptive Router Noise

GARN (Gradient-Adaptive Router Noise) stabilizes the routing dynamics by injecting noise whose magnitude is adapted to the gradient of the routing distribution. The key insight is that collapse is signaled by *decreasing entropy* of π, and the noise should increase precisely when entropy is low.

Definition 9.1 (GARN)**.** Given router logits $\mathbf{z} \in \mathbb{R}^N$, the GARN-perturbed logits are:

$$\tilde{\mathbf{z}}_i = z_i + \sigma_i \cdot \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 1), \tag{43}$$

where the noise scale σ_i is:

$$\sigma_i = \sigma_{\text{base}} \cdot \exp\left(-\eta \cdot H(\pi)\right) \cdot \left(1 + \alpha \cdot |\nabla_{z_i} \mathcal{L}|\right), \tag{44}$$

$H(\pi) = -\sum_i \pi_i \log \pi_i$ is the entropy of the routing distribution, σ_{base} is the base noise level, $\eta > 0$ controls entropy sensitivity, and $\alpha > 0$ controls gradient sensitivity.

The noise scale has three components:

1. **Entropy term** $\exp(-\eta H(\pi))$: When entropy is low (imminent collapse), the noise scale *increases*, perturbing the router away from the degenerate distribution. When entropy is high (balanced routing), the noise scale *decreases*, allowing the router to specialize without interference.
2. **Gradient term** $(1 + \alpha|\nabla_{z_i} \mathcal{L}|)$: Experts with large routing gradients (those whose selection is changing rapidly) receive more noise, preventing hasty commitment to a single expert. In practice, $|\nabla_{z_i} \mathcal{L}|$ is approximated by $|z_i|$ (see Remark 9.1).
3. **Base term** σ_{base}: A minimum noise floor that ensures exploration even when entropy and gradients are small.

Remark 9.1 (Practical Approximation of the Gradient Term)**.** The gradient term $|\nabla_{z_i} \mathcal{L}|$ requires the loss gradient with respect to the router logits, which is unavailable during the forward pass. The reference implementation (Section 15) uses $|z_i|$ as a proxy: since $\nabla_{z_i} \mathcal{L} = \pi_i(\delta_{i,y} - \pi_y)$ for cross-entropy loss, and $\pi_i \propto \exp(z_i)$ for the dominant expert, logit magnitude correlates with gradient magnitude. This proxy introduces a systematic bias but preserves the qualitative behavior (experts with larger logits receive more noise). A more accurate but computationally expensive alternative is a *two-pass GARN*: first forward without noise, then backward to obtain $\nabla_z \mathcal{L}$, then a second forward with gradient-scaled noise at approximately $2\times$ the cost.

9.3 Stability Analysis

Theorem 9.1 (GARN Stabilization)**.** Consider the routing ODE:

$$\frac{d\pi}{dt} = F(\pi) + \sigma(\pi) \cdot \frac{dW}{dt}, \tag{45}$$

where W is a Wiener process and $\sigma(\pi) = \sigma_{\text{base}} \exp(-\eta H(\pi))$. If the deterministic system $\dot{\pi} = F(\pi)$ has an attracting fixed point at the degenerate distribution $\pi^* = \mathbf{e}_i$, then for sufficiently large σ_{base} and η, the stochastic system has a unique stationary distribution ρ^* with $H(\rho^*) > 0$, i.e., the stationary distribution is non-degenerate.

This result follows from the theory of stochastic stabilization (Arnold, 2013): additive noise with entropy-dependent scaling destabilizes the degenerate fixed point while preserving the stability of non-degenerate fixed points. The gradient-adaptive term provides an additional stabilizing mechanism by increasing perturbation precisely where the dynamics are most unstable.

9.4 Fokker–Planck Analysis and Stationary Distribution

Theorem 9.1 can be strengthened by explicitly deriving the stationary distribution of the GARN-driven router via the Fokker–Planck equation. We model the evolution of the routing probability density $p(\pi, t)$ on the probability simplex Δ^{N-1}.

Proposition 9.2 (Fokker–Planck Equation for GARN Dynamics)**.** Let the routing logits $\mathbf{z}$ evolve according to the Itô stochastic differential equation:

$$d\mathbf{z} = F(\mathbf{z})\, dt + \Sigma(\mathbf{z})\, dW_t, \tag{46}$$

where $F(\mathbf{z})$ is the deterministic drift (gradient of the task loss with respect to the logits) and $\Sigma(\mathbf{z}) = \text{diag}(\sigma(\pi))$ is the GARN diffusion matrix with $\sigma_i = \sigma_{\text{base}} \exp(-\eta H(\pi))(1 + \alpha|z_i|)$. By Itô's lemma, the induced SDE for the routing probabilities $\pi = \text{softmax}(\mathbf{z}/\tau)$ yields a Fokker–Planck equation for the density $p(\pi, t)$:

$$\frac{\partial p}{\partial t} = -\nabla_\pi \cdot \left[\tilde{F}(\pi)\, p\right] + \frac{1}{2}\nabla_\pi^2 : \left[\tilde{\Sigma}(\pi)\tilde{\Sigma}(\pi)^\top p\right], \tag{47}$$

where $\tilde{F}$ and $\tilde{\Sigma}$ are the drift and diffusion coefficients transformed to the probability space via the softmax Jacobian.

Remark 9.2 (Boundary Repulsion and Collapse Prevention)**.** The stationary distribution $\rho^*(\pi) = \lim_{t\to\infty} p(\pi, t)$ is the solution of $\partial p/\partial t = 0$. A key property of the GARN diffusion is that near the boundary $\partial\Delta^{N-1}$ (where $\pi_i \to 1$ and $\pi_{j\neq i} \to 0$), the entropy $H(\pi) \to 0$, causing the diffusion coefficient $\sigma(\pi) \propto \exp(-\eta H(\pi))$ to diverge. This creates an effective *repulsive potential* at the boundary of the simplex. Formally, the infinitesimal generator $\mathcal{L}_{\text{diff}}$ of the diffusion process satisfies $\mathcal{L}_{\text{diff}} H(\pi) > 0$ near $\partial\Delta^{N-1}$, which implies that the stationary density vanishes at the boundary: $\rho^*(\pi) \to 0$ as $\pi \to \partial\Delta^{N-1}$. This provides a rigorous guarantee that router collapse—the trapping of the routing distribution at a degenerate vertex of the simplex—is impossible under GARN dynamics. The diffusion term acts as a strongly reflecting boundary condition, ensuring that all experts receive non-zero expected gradient updates.

The Fokker–Planck formalism provides a quantitative tool for analyzing the trade-off between specialization (low entropy, strong deterministic drift) and exploration (high diffusion near degenerate states). The entropy sensitivity η controls the strength of the boundary repulsion: larger η produces stronger repulsion, preventing collapse more aggressively but potentially slowing convergence to useful specializations. The optimal η balances these competing objectives and can in principle be estimated from the spectral properties of the Fokker–Planck operator.

10 Path Consensus and Cross-Layer Coherence

10.1 Motivation

In standard MoE, each layer independently routes tokens to experts. This independence can lead to *representational fragmentation*: a token that is routed to a "syntax" expert in layer 3 might be routed to a "semantics" expert in layer 5, disrupting the coherent processing pathway. PathMoE demonstrated that enforcing cross-layer routing consistency improves both training stability and final model quality.

10.2 Path Consensus Loss

AS-MoE enforces cross-layer coherence through the **Path Consensus Loss**, which penalizes deviations between routing distributions at adjacent layers:

Definition 10.1 (Path Consensus Loss)**.** Let $\pi^{(\ell)}$ and $\pi^{(\ell+1)}$ be the routing distributions (over experts) at layers ℓ and $\ell+1$. The path consensus loss is:

$$\mathcal{L}_{\text{path}} = \frac{1}{L-1} \sum_{\ell=1}^{L-1} D_{\text{KL}} \left(\pi^{(\ell+1)} \parallel \pi^{(\ell)} \right), \tag{48}$$

where D_{KL} is the Kullback-Leibler divergence and L is the total number of MoE layers. The direction $D_{\text{KL}}(\pi^{(\ell+1)} \| \pi^{(\ell)})$ is chosen because minimizing this forces the later distribution $\pi^{(\ell+1)}$ to cover the support of the earlier distribution $\pi^{(\ell)}$ (mean-seeking behavior), effectively regularizing the later layer toward the earlier one.

The KL divergence is asymmetric by design: we regularize the later layer toward the earlier one, under the assumption that early layers establish the representational pathway and later layers should refine rather than overturn it. This is consistent with the observation that early MoE layers tend to capture syntactic patterns while later layers capture semantic patterns, and disrupting this hierarchy is detrimental.

10.3 Soft Constraint via Lagrangian

The path consensus loss is incorporated as a soft constraint with a learnable weight λ_{path}, which is annealed during training:

$$\lambda_{\text{path}}^{(t)} = \lambda_{\text{path}}^{(0)} \cdot \min \left(1, \frac{t}{T_{\text{warmup}}} \right), \tag{49}$$

where T_{warmup} is the warmup period. During early training, the path consensus constraint is weak, allowing the model to discover effective routing patterns. As training progresses, the constraint strengthens, locking in stable expert trajectories.

10.4 Relationship to Curriculum Logic

The Path Consensus Loss complements the Generation curriculum (Section 6). While the curriculum ensures that tokens pass through the Shared Expert before receiving specialized processing, the Path Consensus Loss ensures that this processing trajectory is consistent across layers. Together, they enforce two complementary stability properties:

1. **Vertical stability** (curriculum): Simple processing always precedes complex processing within a layer.
2. **Horizontal stability** (path consensus): Processing pathways are consistent across layers.

11 Mathematical Foundations

11.1 Master Functional Equation

The complete AS-MoE training objective integrates all components into a single functional:

Definition 11.1 (Master Functional)**.** The AS-MoE training objective at step t is:

$$\mathcal{L}^{(t)} = \mathcal{L}_{\text{task}} + \lambda_{\text{cons}}\mathcal{L}_{\text{cons}} + \lambda_{\text{div}}\mathcal{L}_{\text{div}} + \lambda_{\text{ortho}}\mathcal{L}_{\text{ortho}} + \mu^{(t)}\left(\mathcal{B}^{(t)} - B_{\text{target}}\right) + \lambda_{\text{path}}^{(t)}\mathcal{L}_{\text{path}} + \lambda_{\text{AIC}}\mathcal{L}_{\text{AIC}} + \lambda_{\text{replay}}\mathcal{L}_{\text{replay}}, \quad (50)$$

where:

- $\mathcal{L}_{\text{task}}$: primary language modeling loss (cross-entropy).
- $\mathcal{L}_{\text{cons}}$: generation consensus loss (encourages cross-channel agreement on generation level).
- $\mathcal{L}_{\text{div}}$: flavor diversity loss (prevents flavor collapse).
- $\mathcal{L}_{\text{ortho}}$: channel orthogonality loss.
- $\mu^{(t)}(\mathcal{B}^{(t)} - B_{\text{target}})$: Lagrangian budget control.
- $\mathcal{L}_{\text{path}}$: path consensus loss.
- $\mathcal{L}_{\text{AIC}}$: activation-importance consolidation on Shared Expert (Section 14).
- $\mathcal{L}_{\text{replay}}$: routing replay buffer loss (Section 14).

11.2 Convergence of the Full Objective

Theorem 11.1 (Convergence Under Gradient Mixing + GARN)**.** Consider the AS-MoE training dynamics with gradient mixing parameter $\gamma < 1/(N-1)$ (Lemma 8.1) and GARN noise with entropy sensitivity $\eta > 0$ (Theorem 9.1). Under standard assumptions (L-smoothness of $\mathcal{L}_{\text{task}}$, bounded gradient variance, learning rate η_t satisfying Robbins-Monro conditions), the iterates $\boldsymbol{\theta}^{(t)}$ converge to a stationary point of the regularized objective:

$$\mathbb{E}\left[\|\nabla\hat{\mathcal{L}}(\boldsymbol{\theta}^{(t)})\|^2\right] \to 0 \quad \text{as } t \to \infty, \quad (51)$$

where $\hat{\mathcal{L}} = \mathcal{L}_{\text{task}} + \text{regularization terms}$, and the expectation is over both the data distribution and the GARN noise.

Proof Sketch. The proof relies on the framework of two-time-scale stochastic approximation for non-convex optimization.

Step 1: Gradient Mixing forms a valid preconditioner. Let $\mathbf{g}_t = \nabla\hat{\mathcal{L}}(\boldsymbol{\theta}^{(t)})$. The gradient mixing operation applies the matrix $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$. By Lemma 8.1, since $\gamma < 1/(N-1)$, the minimum eigenvalue of $\mathbf{M}$ is strictly bounded away from zero: $\lambda_{\min}(\mathbf{M}) \geq 1 - \gamma(N-1) > 0$. Thus, $\mathbf{M}$ is strictly positive definite, and $-\mathbf{M}\mathbf{g}_t$ provides a valid descent direction: $\langle \mathbf{g}_t, -\mathbf{M}\mathbf{g}_t\rangle \leq -\lambda_{\min}\|\mathbf{g}_t\|^2$.

Step 2: GARN prevents absorption into degenerate states. In standard MoE training, routers can become trapped in absorbing degenerate states on the boundary of the probability simplex $\partial\Delta^{N-1}$ (where $\pi_i = 1$ and $\pi_{j\neq i} = 0$), leading to zero gradients for unselected experts. The GARN stochastic differential equation (Theorem 9.1) introduces a diffusion term $\sigma(\pi)dW_t$. Because $\sigma(\pi) \propto \exp(-\eta H(\pi))$, as $\pi \to \partial\Delta^{N-1}$ (meaning $H(\pi) \to 0$), the diffusion

coefficient exponentially dominates the deterministic drift $F(\pi)$. Let $\mathcal{L}_{\text{diff}}$ be the infinitesimal generator of this process; we have $\mathcal{L}_{\text{diff}} H(\pi) > 0$ near the boundary, functioning as a strongly reflecting boundary condition. This guarantees the stationary measure ρ^* has zero mass on $\partial\Delta^{N-1}$, ensuring all experts receive non-zero expected mini-batch tokens.

Step 3: Two-time-scale Lyapunov convergence. The dynamics of $\boldsymbol{\theta}$ (fast variable) and μ (slow variable) operate on different time scales due to the EMA budget tracking. Following the Borkar paradigm (2008) for stochastic approximation, the tracking estimator $\bar{\mathcal{B}}$ allows the slow variable μ to track the ODE $\dot{\mu} = \text{Proj}_{\geq 0}(\mathcal{B}(\boldsymbol{\theta}(\mu)) - B_{\text{target}})$. Using the control Lyapunov function $V(\boldsymbol{\theta}, \mu) = \hat{\mathcal{L}}(\boldsymbol{\theta}) + \frac{1}{2c}\|\mu - \mu^*\|^2$, the strictly positive eigenvalues from Step 1 and the guaranteed exploration from Step 2 ensure the coupled system descends V in expectation, leading to almost sure convergence to the KKT stationary points of the Lagrangian. □

11.3 Operator-Theoretic View: Core + LoRA as Perturbation

The Core + LoRA factorization can be formalized more precisely using Kato's perturbation theory for bounded linear operators on Hilbert spaces.

Let $\mathcal{H} = \mathbb{R}^d$ be the representation space and $\mathcal{B}(\mathcal{H})$ the space of bounded linear operators on $\mathcal{H}$. The expert operator $T_{g,f,k} \in \mathcal{B}(\mathcal{H})$ acts as:

$$T_{g,f,k}(\mathbf{x}) = \sigma\left(\mathbf{x}(\mathbf{W}_g^{\text{core}} + \mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)})\right)\mathbf{W}_g^{\text{core},\downarrow}, \tag{52}$$

where σ is the activation function and $\mathbf{W}_g^{\text{core},\downarrow}$ is the shared down-projection.

Proposition 11.2 (Perturbation Bound on Linear Part)**.** Let T_g^{core} be the Core linear operator (LoRA terms set to zero, *before* activation σ) and $\Delta T_{g,f,k}$ the LoRA perturbation to the linear part. Then the operator norm of the linear perturbation satisfies:

$$\|\mathbf{W}_g^{\text{core}} + \mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)} - \mathbf{W}_g^{\text{core}}\|_{\text{op}} = \|\mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)}\|_{\text{op}} \leq \|\mathbf{A}^{(g,f,k)}\|_2 \cdot \|\mathbf{B}^{(g,f,k)}\|_2 \cdot \frac{\alpha}{r}. \tag{53}$$

For the full nonlinear expert operator $T_{g,f,k}$ with activation σ (Lipschitz constant L_σ), the Lipschitz constant of the perturbation satisfies:

$$\text{Lip}(T_{g,f,k} - T_g^{\text{core}}) \leq L_\sigma \cdot \|\mathbf{W}_g^{\text{core},\downarrow}\|_{\text{op}} \cdot \|\mathbf{A}^{(g,f,k)}\|_2 \cdot \|\mathbf{B}^{(g,f,k)}\|_2 \cdot \frac{\alpha}{r}. \tag{54}$$

Since $\text{rank}(\mathbf{AB}) = r \ll d$, the perturbation is confined to a low-dimensional subspace, and its spectral impact decays as $O(\alpha/r)$.

Remark 11.1. The bound above applies to the Lipschitz constant of the operator difference, not to the operator norm of the difference of nonlinear operators. A direct bound on $\|T_{g,f,k} - T_g^{\text{core}}\|_{\text{op}}$ would require additional assumptions on the input distribution (e.g., bounded $\|\mathbf{x}\|$). The Lipschitz bound is sufficient to guarantee that the LoRA adaptation does not drastically alter the input-output mapping of the expert.

11.4 Spectral Gap and Optimal LoRA Rank Constraint

To formalize the conjecture regarding the optimal LoRA rank r (identified as an open question in Section 17), we analyze the singular value spectrum of the Core operator $\mathbf{W}^{\text{core}}$. The choice of LoRA rank in standard architectures is typically treated as an empirical hyperparameter. In AS-MoE, Kato's perturbation theory and the Davis–Kahan $\sin\Theta$ theorem provide a theoretical condition for r to preserve the invariant subspaces of the "Higgs vacuum" (Core weights).

Definition 11.2 (Spectral Gap)**.** Let the frozen Core matrix $\mathbf{W}_g^{\text{core}}$ have singular values $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_{d_{\min}}$. The spectral gap at rank r is defined as $\delta_r = \sigma_r - \sigma_{r+1}$. It quantifies the separation between the r-th dominant feature subspace and the residual noise subspace of the shared Core geometry.

Theorem 11.3 (Spectral Isolation and Subspace Preservation)**.** Let the singular values of the linear core operator T_g^{core} be $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_d$. Suppose the LoRA rank r is chosen such that there is a dominant spectral gap $\delta = \sigma_r - \sigma_{r+1} > 0$. Let $V_r \subset \mathcal{H}$ be the invariant subspace spanned by the top r right singular vectors. Let $\tilde{V}_r$ be the corresponding perturbed subspace under the LoRA adaptation $\Delta T_{g,f,k} = \mathbf{AB}$. By the Davis–Kahan $\sin \mathbf{\Theta}$ theorem, if the perturbation norm satisfies $\|\mathbf{AB}\|_{\text{op}} < \delta/2$, then the maximal principal angle $\theta_{\max}$ between the original and perturbed subspaces is strictly bounded:

$$\|\sin \mathbf{\Theta}(V_r, \tilde{V}_r)\|_{\text{op}} \leq \frac{\|\mathbf{AB}\|_{\text{op}}}{\delta - \|\mathbf{AB}\|_{\text{op}}} \leq \frac{\|\mathbf{A}\|_2 \|\mathbf{B}\|_2 \cdot \frac{\alpha}{r}}{\delta - \|\mathbf{A}\|_2 \|\mathbf{B}\|_2 \cdot \frac{\alpha}{r}}. \tag{55}$$

Proof. By Weyl's inequality for singular values, the perturbed singular values $\tilde{\sigma}_i$ of the combined expert matrix satisfy:

$$|\tilde{\sigma}_i - \sigma_i| \leq \|\Delta \mathbf{W}\|_2 \quad \text{for all } i. \tag{56}$$

For the invariant subspace spanned by the top r singular vectors to remain strictly disjoint from the remaining spectrum (preserving the fundamental representational topology of the Core), we require the perturbed gap to remain strictly positive: $\tilde{\sigma}_r - \tilde{\sigma}_{r+1} > 0$. Substituting the worst-case Weyl bounds yields:

$$(\sigma_r - \|\Delta \mathbf{W}\|_2) - (\sigma_{r+1} + \|\Delta \mathbf{W}\|_2) > 0 \implies \|\Delta \mathbf{W}\|_2 < \frac{\sigma_r - \sigma_{r+1}}{2}. \tag{57}$$

Since $\Delta \mathbf{W}$ is explicitly parameterized as scaled matrices $\frac{\alpha}{r}\mathbf{AB}$, substituting this norm and applying the Davis–Kahan theorem completes the proof. □

Remark 11.2 (Resolution of the Optimal LoRA Rank Conjecture)**.** Theorem 11.3 directly resolves the open question regarding the optimal LoRA rank (Section 17). The rank r should not be an arbitrary hyperparameter, but should be aligned with the index of the maximal spectral gap δ in the pre-trained Core weights. As long as the LoRA scaling α/r restricts the adapter's capacity below the $\delta/2$ threshold, the primary representational "coordinate system" (defined by V_r) of the expert persists. This provides a rigorous mathematical guarantee for safe domain transfer without destroying the foundational "Higgs vacuum" structure. In practice, this means that the optimal r can be determined algorithmically: compute the SVD of $\mathbf{W}_g^{\text{core}}$, identify the largest gap δ_r in the singular value spectrum, and set the LoRA rank to r.

11.5 Algorithmic Formalization

To consolidate the mathematical definitions presented above into actionable procedures, we provide the formal pseudocodes for the core AS-MoE mechanisms. Algorithm 1 details the forward pass incorporating the Core + LoRA factorization and GARN. Algorithm 2 formalizes the Covariance Gradient Mixing in the backward pass, and Algorithm 3 specifies the two-phase Lagrangian budget controller.

Input: Token $\mathbf{x}$, Channel projections $\mathbf{W}_k$, Routers $\mathbf{W}_g, \mathbf{W}_f^{(g,k)}$
Output: Final representation $\mathbf{y}$, Auxiliary loss tuple $\mathcal{L}_{\text{aux}}$
Initialize $\mathbf{y} \leftarrow \mathbf{0}$;
for *channel* $k = 1$ **to** K **do**
  $\mathbf{h}_k = \mathbf{x}\mathbf{W}_k$; `/* Multi-view projection */`
  `/* 1. Generation Routing with GARN */`
  $\mathbf{z}_g = \mathbf{h}_k \mathbf{W}_g + \boldsymbol{\phi}_g$;
  $\pi_g^{\text{raw}} = \text{softmax}(\mathbf{z}_g)$;
  $\sigma_g = \sigma_{\text{base}} \cdot \exp\left(-\eta H(\pi_g^{\text{raw}})\right) \odot (1 + \alpha|\mathbf{z}_g|)$;
  $\tilde{\mathbf{z}}_g = \mathbf{z}_g + \sigma_g \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$;
  $\pi_g = \text{softmax}(\tilde{\mathbf{z}}_g / \tau_g)$;
  $g^* = \text{argmax}_g(\pi_g)$; `/* Hard assignment (STE in backward) */`
  `/* 2. Curriculum Logic: Compute up to` g^* `*/`
  $\mathbf{y}_k \leftarrow \mathbf{0}$;
  for *generation* $g = 0$ **to** g^* **do**
    `/* 3. Flavor Routing with GARN */`
    $\mathbf{z}_f = \mathbf{h}_k \mathbf{W}_f^{(g,k)} + \boldsymbol{\phi}_f^{(g,k)}$;
    Apply GARN perturbation to $\mathbf{z}_f \to \tilde{\mathbf{z}}_f$;
    $\pi_f^{(g,k)} = \text{softmax}(\tilde{\mathbf{z}}_f / \tau_f)$;
    $\mathcal{S}_f = \text{top-}m(\pi_f^{(g,k)})$; `/* Select active flavors */`
    for $f \in \mathcal{S}_f$ **do**
      `/* 4. Core + LoRA Expert Computation */`
      $\mathbf{W}_{\text{up}} = \mathbf{W}_{\text{up},g}^{\text{core}} + \frac{\alpha}{r}\mathbf{A}_{\text{up}}^{(g,f,k)}\mathbf{B}_{\text{up}}^{(g,f,k)}$;
      $\mathbf{W}_{\text{down}} = \mathbf{W}_{\text{down},g}^{\text{core}} + \frac{\alpha}{r}\mathbf{A}_{\text{down}}^{(g,f,k)}\mathbf{B}_{\text{down}}^{(g,f,k)}$;
      $\mathbf{y}_k \leftarrow \mathbf{y}_k + \pi_f^{(g,k)} \cdot \sigma(\mathbf{h}_k \mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$;
    end
  end
  $\mathbf{y} \leftarrow \mathbf{y} + \frac{1}{K}\mathbf{y}_k$; `/* Average over channels */`
end
Compute $\mathcal{L}_{\text{cons}}, \mathcal{L}_{\text{div}}, \mathcal{L}_{\text{ortho}}$ from accumulated $\{\pi_g, \pi_f^{(g,k)}, \mathbf{W}_k\}$;
return $\mathbf{y}, (\mathcal{L}_{\text{cons}}, \mathcal{L}_{\text{div}}, \mathcal{L}_{\text{ortho}})$

Algorithm 1: AS-MoE Forward Pass with GARN

Input: Raw gradients $\{\nabla^{\text{raw}}_{\mathbf{y}_i}\}_{i=1}^N$, Expert outputs $\{\mathbf{y}_i\}_{i=1}^N$ in channel k, Mixing strength γ
Output: Mixed gradients $\{\nabla^{\text{mixed}}_{\mathbf{y}_i}\}_{i=1}^N$
`/* 1. Compute pairwise activation correlations */`
for $i = 1$ **to** N **do**
  for $j = 1$ **to** N **do**
    $R_{ij} = \frac{\langle \mathbf{y}_i, \mathbf{y}_j \rangle}{\|\mathbf{y}_i\|_2 \cdot \|\mathbf{y}_j\|_2}$;
  end
end
`/* 2. Construct gluon-exchange mixing matrix */`
$\mathbf{M} = \mathbf{I} + \gamma \mathbf{R}$;
`/* 3. Distribute gradients among correlated experts */`
for $i = 1$ **to** N **do**
  $\nabla^{\text{mixed}}_{\mathbf{y}_i} = \sum_{j=1}^N M_{ji} \cdot \nabla^{\text{raw}}_{\mathbf{y}_j}$;
end
return $\{\nabla^{\text{mixed}}_{\mathbf{y}_i}\}_{i=1}^N$

Algorithm 2: Covariance Gradient Mixing (Backward Pass)

Input: Actual hard routing assignments for batch, Target budget B_{target}, Previous EMA $\bar{\mathcal{B}}^{(t-1)}$, Previous dual $\mu^{(t-1)}$
Output: Updated dual $\mu^{(t)}$, Budget loss penalty $\mathcal{L}_{\text{budget}}$
`/* Phase A: Empirical Budget Estimation (EMA) */`
Compute hard FLOPs for batch: $\mathcal{B}^{(t)}_{\text{batch}} = \frac{1}{|\text{batch}|} \sum_{b \in \text{batch}} \sum_{g=0}^{g_b^*} B^{(g)}$;
Update EMA: $\bar{\mathcal{B}}^{(t)} \leftarrow \beta \cdot \bar{\mathcal{B}}^{(t-1)} + (1-\beta) \cdot \mathcal{B}^{(t)}_{\text{batch}}$;
`/* Phase B: Dual Ascent Step */`
Calculate budget violation: $\Delta = \bar{\mathcal{B}}^{(t)} - B_{\text{target}}$;
Update multiplier (projected onto $\mathbb{R}_{\geq 0}$): $\mu^{(t)} = \max\left(0,\ \mu^{(t-1)} + \eta_\mu \cdot \Delta\right)$;
`/* Phase C: Primal Penalty Construction */`
Note: The primal step uses the pre-update dual variable to decouple the min-max dynamics, avoiding synchronous oscillations.
$\mathcal{L}_{\text{budget}} = \mu^{(t-1)} \cdot \Delta$;
return $\mu^{(t)}, \mathcal{L}_{\text{budget}}$

Algorithm 3: Lagrangian Two-Phase Budget Controller

12 Ontogenic Pre-training and Category-Theoretic Bridge

12.1 From "Zero Quarks" to Language

A distinctive feature of AS-MoE is the **ontogenic pre-training** protocol, which begins not with language data, but with synthetic "quark-level" attributes—elementary properties that compose to form the representational primitives of language. The motivation is two-fold:

1. **Structural bootstrapping**: By first training the expert ensemble on well-structured synthetic data, we establish a functional routing topology (which experts specialize in which attribute combinations) before exposing the model to the noise and ambiguity of natural language.

2. **Compositional transfer**: The attributes are designed to compose hierarchically, mirroring the compositional structure of language (phonemes → morphemes → words → phrases). This compositional prior is formally captured by the category-theoretic framework below.

12.2 Phase 0: Synthetic Quarks

In Phase 0, the model is trained on synthetic data where each input is a bag of discrete attributes ("quarks"):

$$\mathbf{x}_{\text{quark}} = \text{Embed}(a_1) \oplus \text{Embed}(a_2) \oplus \cdots \oplus \text{Embed}(a_m), \tag{58}$$

where $a_i \in \mathcal{A}$ are attributes from a predefined vocabulary $\mathcal{A}$, and $\oplus$ denotes summation-based aggregation: $\oplus_i \text{Embed}(a_i) = \frac{1}{m}\sum_{i=1}^{m} \text{Embed}(a_i)$. (Alternative aggregation strategies such as attention-weighted combination are discussed in Section 17.)

The training objective in Phase 0 is attribute prediction:

$$\mathcal{L}_{\text{Phase 0}} = -\sum_{i=1}^{m} \log P(a_i \mid \mathbf{x}_{\text{quark}}). \tag{59}$$

This objective forces the routing topology to organize around attribute-level structure: specific flavors specialize in specific attribute types, and the generation hierarchy reflects attribute complexity.

12.3 Phase 1: Language Immersion

In Phase 1, the model transitions to language data. The Core weights from Phase 0 are frozen, and only the LoRA adapters and routers are trained. An **inductive bias loss** encourages the Phase 1 routing to be consistent with the Phase 0 attribute structure:

$$\mathcal{L}_{\text{IB}} = D_{\text{KL}}\left(\pi_{\text{lang}} \parallel \pi_{\text{quark}}^{(\text{frozen})}\right), \tag{60}$$

where $\pi_{\text{quark}}^{(\text{frozen})}$ is the routing distribution from Phase 0 (used as a fixed prior) and π_{lang} is the Phase 1 routing distribution.

12.4 Category-Theoretic Formalization

The compositional bridge from quarks to language can be formalized as a **functor** between categories.

Definition 12.1 (Quark Category $\mathbf{C}_{\text{quark}}$)**.** The quark category has:

- **Objects**: Finite multisets of attributes $[a_1, a_2, \ldots, a_m]$, where $a_i \in \mathcal{A}$.
- **Morphisms**: Attribute-preserving maps $f : [a_i] \to [b_j]$ such that $[b_j]$ can be obtained from $[a_i]$ by adding, removing, or substituting attributes.

$\mathbf{C}_{\text{quark}}$ has finite coproducts (disjoint union of attribute multisets).

Definition 12.2 (Language Category $\mathbf{C}_{\text{lang}}$)**.** The language category has:

- **Objects**: Token sequences $[t_1, t_2, \ldots, t_n]$, where $t_i \in \mathcal{V}$ (vocabulary).
- **Morphisms**: Context-preserving maps (e.g., synonym substitution, paraphrase).

$\mathbf{C}_{\text{lang}}$ has finite coproducts (concatenation of token sequences).

Theorem 12.1 (Compositional Bridge)**.** There exists a functor $\mathcal{F} : \mathbf{C}_{\text{quark}} \to \mathbf{C}_{\text{lang}}$ that:

1. Preserves coproducts: $\mathcal{F}(A \sqcup B) \cong \mathcal{F}(A) \sqcup \mathcal{F}(B)$.
2. Maps each attribute $a \in \mathcal{A}$ to a distribution over token sequences in $\mathcal{V}$.

3. Preserves the compositionality of attributes: the representation of a composite quark is determined by the representations of its constituent attributes.

The inductive bias loss $\mathcal{L}_{\text{IB}}$ is the practical instantiation of the functor constraint: it ensures that the routing topology learned in Phase 0 (over quarks) is approximately preserved in Phase 1 (over language).

The functorial perspective provides a formal justification for why ontogenic pre-training should improve sample efficiency: the functor $\mathcal{F}$ is learned in Phase 0 from the well-structured quark data, and Phase 1 only needs to refine $\mathcal{F}$ on the noisier language data rather than learning it from scratch. The coproduct preservation property ensures that compositional structure is maintained across the bridge.

12.5 Domain Transfer as a Natural Transformation

The true power of the functorial formalism emerges when considering domain transfer (Phase 2). Let $\mathcal{F}_{\text{src}}$ be the functor mapping quarks to the source domain (e.g., general English) and $\mathcal{F}_{\text{tgt}}$ be the target domain (e.g., medical text or code). We conjecture that the Core + LoRA factorization explicitly models a **natural transformation** $\eta : \mathcal{F}_{\text{src}} \Rightarrow \mathcal{F}_{\text{tgt}}$.

For every object $Q \in \mathbf{C}_{\text{quark}}$, there exists a morphism $\eta_Q : \mathcal{F}_{\text{src}}(Q) \to \mathcal{F}_{\text{tgt}}(Q)$ such that for any quark morphism $f : Q_1 \to Q_2$, the following diagram commutes:

$$\begin{tikzcd}
\mathcal{F}_{\text{src}}(Q_1) \arrow[r, "\eta_{Q_1}"] \arrow[d, "\mathcal{F}_{\text{src}}(f)"'] & \mathcal{F}_{\text{tgt}}(Q_1) \arrow[d, "\mathcal{F}_{\text{tgt}}(f)"] \\
\mathcal{F}_{\text{src}}(Q_2) \arrow[r, "\eta_{Q_2}"'] & \mathcal{F}_{\text{tgt}}(Q_2)
\end{tikzcd}$$

In the context of AS-MoE:

- The frozen **Core weights** encode the vertical morphisms $\mathcal{F}_{\text{src}}(f)$ and $\mathcal{F}_{\text{tgt}}(f)$ (the structurally invariant part of the language that is shared across domains).
- The trainable **LoRA adapters** learn the horizontal morphisms η_Q (the domain-specific vocabulary and stylistic translations between source and target representations).

Because η is a natural transformation, local adaptations learned via LoRA systematically cohere across the entire compositional structure of the language. This provides a rigorous mathematical explanation for why low-rank adaptations transfer so efficiently compared to unstructured fine-tuning: the naturality condition ensures that the adaptation is *consistent* across all compositional contexts, not merely locally optimal at each point. Violations of naturality would manifest as inconsistent expert behavior across different input contexts—precisely the failure mode observed in unstructured fine-tuning where experts can develop contradictory specializations.

12.6 Sample Complexity of the Compositional Functor

A primary motivation for Phase 0 ontogenic pre-training is statistical efficiency. We theoretically formalize this by bounding the number of synthetic quark samples needed to learn an ϵ-approximation of the functor $\mathcal{F}$.

Theorem 12.2 (Sample Complexity of Topological Bootstrapping)**.** Let $\mathcal{H}_{\text{route}}$ be the hypothesis class of the AS-MoE routing network (generating the topological geometry over $\mathbf{C}_{\text{quark}}$). Assuming the true structural functor $\mathcal{F}$ is realizable within $\mathcal{H}_{\text{route}}$, establishing a routing topology with expected error $\mathbb{E}[\mathcal{L}_{\text{Phase 0}}(\hat{\mathcal{F}})] \leq \epsilon$ with probability minimum $1-\delta$ requires N synthetic samples, where:

$$N \geq C \frac{d_{\text{VC}}(\mathcal{H}_{\text{route}}) \log(1/\epsilon) + \log(1/\delta)}{\epsilon^2}, \tag{61}$$

Here, $C > 0$ is a universal constant, and $d_{\text{VC}}(\mathcal{H}_{\text{route}}) = \mathcal{O}(N_e \cdot d_{\text{proj}} \log d_{\text{proj}})$ is the VC-dimension of the multi-channel routers.

Remark 12.1 (Efficiency Gap)**.** Crucially, $d_{\text{VC}}(\mathcal{H}_{\text{route}})$ depends only on the router projection dimension (d_{proj}) and the number of experts (N_e), and is entirely independent of the full model dimension d_{model} and FFN sizes d_{ff}. The VC-dimension of the full language model $\mathcal{H}_{\text{LLM}}$ is vastly larger ($d_{\text{VC}}(\mathcal{H}_{\text{route}}) \ll d_{\text{VC}}(\mathcal{H}_{\text{LLM}})$). Theorem 12.2 rigorously proves that structural routing topology can be learned organically with exponentially fewer samples than end-to-end language modeling, justifying the two-phase ontogenic objective. Phase 0 bootstraps the routing structure cheaply (on synthetic data with low VC complexity), and Phase 1 only needs to learn the representational content (Core + LoRA weights) on language data, without simultaneously discovering the routing topology from scratch.

13 Systems Design: CUDA and Data-Oriented Design

13.1 Data-Oriented Design Principles

The AS-MoE architecture is designed from the ground up for efficient GPU execution, following Data-Oriented Design (DoD) principles. The key insight is that the Generation/Flavor/Channel topology maps naturally to a **Structure of Arrays (SoA)** memory layout, where all parameters of the same type are stored contiguously regardless of which expert they belong to.

Table 2: Memory layout comparison: AoS vs. SoA for AS-MoE parameters.

Parameter	**AoS (traditional)**	**SoA (DoD)**
Core up-weights	Scattered per expert	$\mathbf{W}_{\text{up}}^{\text{core}} \in \mathbb{R}^{G \times d \times d_{\text{ff}}^{(\max)}}$ (contiguous)
LoRA **A** matrices	Scattered per expert	$\mathbf{A} \in \mathbb{R}^{G \times F \times K \times d \times r}$ (contiguous)
LoRA **B** matrices	Scattered per expert	$\mathbf{B} \in \mathbb{R}^{G \times F \times K \times r \times d'}$ (contiguous)
Router weights	Per-layer, per-type	$\mathbf{W}_g \in \mathbb{R}^{L \times d \times G}$, $\mathbf{W}_f \in \mathbb{R}^{L \times K \times d \times F}$ (contiguous)

The SoA layout enables:

1. **Coalesced memory access**: All reads of the same parameter type (e.g., all Core up-weights) are contiguous in memory, maximizing GPU memory bandwidth utilization.
2. **Batched operations**: Grouped GEMM can process all experts of the same generation in a single kernel launch.
3. **Efficient all-to-all**: In distributed training, the SoA layout simplifies the all-to-all communication pattern: each GPU sends/receives contiguous slices of the parameter arrays.

13.2 Fused Grouped GEMM

The key computational kernel in AS-MoE is the **Grouped GEMM**, which performs batched matrix multiplication across expert groups with varying dimensions. Unlike standard batched

GEMM (which requires uniform dimensions), Grouped GEMM handles the heterogeneous $d_{\text{ff}}^{(g)}$ across generations:

```
# Conceptual Fused Grouped GEMM kernel
# All generations processed in a single kernel launch
# SoA layout: weights[generation] has shape
#   (num_experts_in_gen, d_model, d_ff_gen)
def fused_grouped_gemm(
    x,            # (tokens, d_model)
    weights,      # SoA: list of (experts, d, d_ff_g)
    route_map,    # (tokens,) -> (generation, flavor, channel)
    active_mask   # (tokens, experts) sparse
):
    outputs = []
    for g in range(G):
        # Gather tokens routed to generation g
        token_idx = (route_map[:, 0] == g)
        x_g = x[token_idx]  # (tokens_g, d_model)
        # Batched GEMM for all (f,k) in generation g
        # All experts in same generation share d_ff
        out_g = grouped_gemm(
            x_g,
            weights[g],        # (F*K, d, d_ff_g)
            active_mask[token_idx, g*F*K:(g+1)*F*K]
        )
        outputs.append(out_g)
    return scatter_combine(outputs, route_map)
```

The fusion eliminates kernel launch overhead (which can dominate runtime for small expert groups) and enables shared memory reuse across the batched GEMM operations. On NVIDIA H100 GPUs, fused Grouped GEMM achieves >90% of peak compute utilization for expert groups of ≥ 8 experts.

13.3 Persistent Kernels: FlashDMoE Integration

For large-scale distributed training, AS-MoE leverages the **FlashDMoE** persistent kernel design. Traditional MoE implementations launch separate kernels for:

1. Dispatch (all-to-all send + local token rearrangement).
2. Expert computation (GEMM + activation).
3. Combine (all-to-all receive + output assembly).

Each kernel launch incurs overhead ($\sim$5–10μs on H100), which accumulates across the L MoE layers in a forward pass. FlashDMoE's persistent kernel design keeps a single kernel resident on the GPU, processing all three phases without re-launching:

$$\text{Latency}_{\text{persistent}} \approx \frac{\text{FLOPs}}{\text{Peak TFLOP/s}} + T_{\text{comm}}, \tag{62}$$

whereas the traditional approach adds $3L \times T_{\text{launch}}$ to the critical path.

13.4 Fused Computation-Communication via NVSHMEM

For multi-node training, AS-MoE employs **NVSHMEM** (NVIDIA Shared Memory) to overlap computation and communication. The key technique is *expert prefetching*: while the current layer's experts are computing, the next layer's expert weights are asynchronously fetched from remote GPUs.

```
# Fused compute-communicate pattern
def fused_layer_forward(x, layer_idx, nvshmem_handle):
    # Launch async fetch for next layer's weights
    if layer_idx < L - 1:
```

```
        nvshmem_handle.prefetch(
            weights[layer_idx + 1],
            stream=cuda_stream_comm)

    # Compute current layer on default stream
    output = asmoe_layer_forward(x, layer_idx)

    # Synchronize: ensure prefetch is complete
    if layer_idx < L - 1:
        cuda_stream_comm.synchronize()

    return output
```

NVSHMEM provides remote direct memory access (RDMA) between GPUs, bypassing the CPU and reducing communication latency by 30–50% compared to NCCL-based all-to-all. The prefetching pattern hides communication latency behind computation, achieving near-zero communication overhead when compute time exceeds communication time (which holds for typical expert dimensions).

14 Training Methodology

14.1 Three-Phase Training Protocol

AS-MoE training proceeds in three phases:

14.1.1 Phase 0: Ontogenic Pre-training (Quark-Level)

1. Generate synthetic attribute data (Section 12).
2. Train the full AS-MoE model on attribute prediction $\mathcal{L}_{\text{Phase 0}}$.
3. Use Expert Choice routing for maximum load balance.
4. Train until routing topology stabilizes (measured by entropy of per-flavor assignment convergence).
5. **Outcome**: A stable routing topology where flavors correspond to attribute types and generations correspond to attribute complexity.

14.1.2 Phase 1: Language Immersion

1. Freeze Core weights from Phase 0.
2. Switch to language data with standard LM objective.
3. Maintain inductive bias loss $\mathcal{L}_{\text{IB}}$ (Section 12).
4. Transition from Expert Choice to Token Choice routing with a warmup schedule.
5. Enable GARN noise injection.
6. Enable gradient mixing.
7. **Outcome**: A language-capable model whose routing topology is grounded in compositional attribute structure.

14.1.3 Phase 2: Domain Transfer (Optional)

1. Freeze Core weights.
2. Re-initialize LoRA adapters for the target domain.
3. Re-initialize routers (allowing new routing patterns).
4. Apply EWC++ to Shared Expert (see below).
5. Apply routing replay buffer (see below).
6. **Outcome**: A domain-specialized model that retains the compositional routing structure from pre-training.

14.2 Anti-Forgetting: Activation-Importance Consolidation (AIC)

Standard Elastic Weight Consolidation (EWC) prevents catastrophic forgetting by penalizing changes to parameters deemed important for the source domain. AS-MoE replaces EWC with **Activation-Importance Consolidation (AIC)**, which measures parameter importance via the sensitivity of *activations* rather than the classical Fisher information on parameters, and applies it selectively to the Shared Expert (g=0):

Definition 14.1 (Activation-Importance Consolidation)**.** The AIC regularization loss is:

$$\mathcal{L}_{\text{AIC}} = \sum_{i \in \mathcal{I}_{g=0}} I_i^{(\text{act})} \left(\theta_i - \theta_i^{\text{pre}}\right)^2, \tag{63}$$

where $\mathcal{I}_{g=0}$ is the set of Shared Expert parameter indices, θ_i^{pre} are the pre-trained values, and $I_i^{(\text{act})}$ is the *activation importance* of parameter θ_i:

$$I_i^{(\text{act})} = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_{\text{source}}} \left[\left(\frac{\partial \log p(\mathbf{x}|\boldsymbol{\theta})}{\partial a_i} \right)^2 \right], \tag{64}$$

where a_i is the activation (pre- or post-nonlinearity) corresponding to parameter θ_i. Unlike the diagonal Fisher information $F_i = \mathbb{E}[(\partial \log p / \partial \theta_i)^2]$, which measures sensitivity of the log-likelihood to parameter changes, $I_i^{(\text{act})}$ measures how strongly the log-likelihood depends on the *activation* modulated by θ_i, thereby capturing the context-dependent importance of θ_i in MoE settings where the same parameter has different effective importance depending on which experts are active.

The selective application to the Shared Expert (g=0) is justified by Proposition 4.1: the Shared Expert processes every token, making it the most critical component for retaining source-domain knowledge. Routed experts (higher generations) are expected to adapt to the new domain, so constraining them is counterproductive.

14.3 Routing Replay Buffer

To prevent the router from "forgetting" how to route source-domain tokens during transfer, AS-MoE maintains a **Routing Replay Buffer** that stores the routing statistics of the source domain:

Definition 14.2 (Routing Replay Buffer)**.** The replay buffer stores:

$$\mathcal{B}_{\text{replay}} = \left\{ \left(\boldsymbol{\mu}_\pi^{(\ell)}, \boldsymbol{\Sigma}_\pi^{(\ell)} \right) \right\}_{\ell=1}^{L}, \tag{65}$$

where $\boldsymbol{\mu}_\pi^{(\ell)} \in \mathbb{R}^{N_e}$ is the mean routing distribution at layer ℓ (averaged over source-domain tokens) and $\boldsymbol{\Sigma}_\pi^{(\ell)} \in \mathbb{R}^{N_e \times N_e}$ is the covariance of the routing distribution. During transfer training, the replay loss is:

$$\mathcal{L}_{\text{replay}} = \sum_{\ell=1}^{L} \left[\|\boldsymbol{\mu}_\pi^{(\ell,t)} - \boldsymbol{\mu}_\pi^{(\ell,\text{src})}\|^2 + \lambda_\Sigma \|\boldsymbol{\Sigma}_\pi^{(\ell,t)} - \boldsymbol{\Sigma}_\pi^{(\ell,\text{src})}\|_F^2 \right]. \tag{66}$$

The covariance term is essential: matching only the mean routing distribution would allow the router to concentrate all variance on a single expert (low entropy), which the covariance term prevents. The replay buffer is lightweight, storing only $O(L \cdot N_e^2)$ values, which is negligible compared to the model parameters.

14.4 Training Recipe: Hyperparameters and Schedules

The preceding subsections describe *what* each training phase does. This subsection specifies *how* to execute each phase concretely: the optimizer settings, learning rate schedules, loss weights, and the precise schedule for enabling and disabling each architectural component. Table 3 provides the complete hyperparameter specification for AS-MoE-M.

Several design choices in this table require justification. The **higher learning rate for routers** (10^{-3} vs. 3×10^{-4} for weights) reflects the observation that routing decisions are discrete and require more aggressive exploration during early training to escape degenerate initializations. The **disabled GARN and gradient mixing in Phase 0** is deliberate: ontogenic pre-training uses Expert Choice routing, which provides perfect load balance by construction, so the collapse-prevention mechanisms are unnecessary and would add noise to the attribute learning signal. The **looser budget target in Phase 0** ($B_{\text{target}} = 2.5$ vs. 2.0) allows the model more flexibility to explore the expert topology before the stricter Phase 1 budget constraint is imposed.

The **Expert Choice → Token Choice transition** is the most critical schedule in the training pipeline. During the first 20K steps of Phase 1, the routing strategy linearly interpolates from Expert Choice (where experts select their top tokens) to Token Choice (where tokens select their top experts). The interpolation is controlled by a mixing coefficient $\alpha_{\text{EC}} \in [0, 1]$ that decays from 1.0 to 0.0:

$$\pi_{\text{final}} = \alpha_{\text{EC}} \cdot \pi_{\text{EC}} + (1 - \alpha_{\text{EC}}) \cdot \pi_{\text{TC}}, \quad \alpha_{\text{EC}} = \max\left(0, 1 - \frac{t}{T_{\text{warmup}}}\right), \tag{67}$$

where t is the current step and $T_{\text{warmup}} = 20\text{K}$. This prevents the sharp distributional shift that would occur from an abrupt switch, which we have empirically observed causes temporary routing instability (entropy drops of 30–50%) lasting 5–10K steps.

14.4.1 Component Enable/Disable Schedule

Table 4 summarizes when each architectural component is active across the three training phases. This is the "switchboard" that a practitioner needs to implement the training loop correctly.

14.5 Initialization Recipe

Correct initialization is critical for AS-MoE. The Core + LoRA factorization means that at initialization, the LoRA matrices $\mathbf{B}$ are zero, so all experts in the same generation produce identical outputs. This is by design—the model starts as a set of identical copies and gradually differentiates through training. The initialization procedure for each parameter group is specified below.

The **channel projection initialization** deserves special attention. Rather than using independent random matrices for $\mathbf{W}_1, \ldots, \mathbf{W}_K$, we generate a single random matrix $\mathbf{Q} \in \mathbb{R}^{d \times d}$

Table 3: Complete training hyperparameters for AS-MoE-M (d_{model}=4096, G=3, F=4, K=2). Phase 0 uses synthetic attribute data; Phase 1 uses language data; Phase 2 uses domain-specific data.

Hyperparameter	**Phase 0**	**Phase 1**	**Phase 2**
Optimizer			
Optimizer	AdamW	AdamW	AdamW
β_1, β_2	0.9, 0.95	0.9, 0.95	0.9, 0.95
Peak LR (Core)	3×10^{-4}	*frozen*	*frozen*
Peak LR (LoRA)	3×10^{-4}	1×10^{-4}	5×10^{-5}
Peak LR (routers)	1×10^{-3}	3×10^{-4}	3×10^{-4}
Peak LR (projections)	3×10^{-4}	1×10^{-4}	5×10^{-5}
Weight decay	0.1	0.1	0.1
Gradient clipping	1.0	1.0	1.0
LR Schedule			
Warmup steps	2000	2000	500
Schedule	Cosine decay	Cosine decay	Linear decay
Min LR ratio	0.1	0.1	0.0
Batch and Compute			
Batch size (tokens)	4M	4M	2M
Total steps	50K	300K	50K
Precision	bf16	bf16	bf16
Routing			
Strategy	Expert Choice	EC $\rightarrow$ TC transition	Token Choice
EC $\rightarrow$ TC warmup	—	20K steps	—
Top-m flavors	1	1	1
Gen temperature τ_g	1.0	1.0 $\rightarrow$ 0.5	0.5
Flavor temperature τ_f	1.0	1.0	1.0
GARN			
σ_{base}	0.0 (disabled)	0.1	0.05
η (entropy scaling)	—	2.0	2.0
α (grad-adaptive)	—	0.5	0.5
Gradient Mixing			
γ (mix strength)	0.0 (disabled)	0.1	0.05
Mixing scope	—	Per-channel	Per-channel
Budget Control			
B_{target}	2.5 (loose)	2.0 (target)	2.0
β_{EMA}	0.99	0.99	0.99
η_μ (dual step size)	0.001	0.01	0.01
Initial μ	0.0	from Phase 0	from Phase 1
Loss Weights			
λ_{budget}	1.0	1.0	1.0
λ_{cons}	0.1	0.1	0.1
λ_{div}	0.01	0.01	0.01
λ_{ortho}	0.1	0.1	0.1
λ_{path}	0.0	0.01	0.01
λ_{IB}	1.0	0.1	0.0
λ_{AIC}	—	—	1000
λ_{replay}	—	—	0.1

Table 4: Component activation schedule across training phases. "Ramp" means the component is enabled with a linear warmup over the first 5K steps of the phase. "Inherit" means the value is carried over from the previous phase.

Component	**Phase 0**	**Phase 1**	**Phase 2**
Core weights	Trainable	**Frozen**	**Frozen**
LoRA $(\mathbf{A}, \mathbf{B})$	Trainable	Trainable	Re-init, trainable
Routers	Trainable	Trainable	Re-init, trainable
Channel projections	Trainable	Trainable	Trainable
Expert Choice routing	**On**	Ramp $\rightarrow$ Off	Off
Token Choice routing	Off	Ramp $\rightarrow$ **On**	**On**
GARN noise	Off	Ramp on	On (reduced)
Gradient mixing (γ)	Off (γ=0)	Ramp on (γ=0.1)	On (γ=0.05)
Budget control (μ)	Loose (B=2.5)	Target (B=2.0)	Inherit
Path consensus	Off	Ramp on	On
Inductive bias loss $\mathcal{L}_{\text{IB}}$	**On** (λ=1.0)	Reduced (λ=0.1)	Off
AIC regularization	Off	Off	**On**
Routing replay buffer	Off	Off	**On**

Table 5: Initialization scheme for all AS-MoE parameters. "Kaiming" refers to He initialization with std $= \sqrt{2/n_{\text{in}}}$, appropriate for layers followed by ReLU/GELU.

Parameter	**Init**	**Rationale**
$\mathbf{W}_{\text{up}}^{\text{core},g}$	Kaiming	Standard FFN init; ensures non-degenerate initial representations
$\mathbf{W}_{\text{down}}^{\text{core},g}$	Kaiming	Same as up-projection
$\mathbf{A}_{\text{up}}^{(g,f,k)}$	Kaiming	Per IniLoRA analysis; preserves forward signal magnitude
$\mathbf{B}_{\text{up}}^{(g,f,k)}$	**Zero**	At init: expert = Core only; LoRA contribution is zero
$\mathbf{A}_{\text{down}}^{(g,f,k)}$	Kaiming	Same as up-projection LoRA
$\mathbf{B}_{\text{down}}^{(g,f,k)}$	**Zero**	Same rationale; delta starts at zero
$\mathbf{W}_g^{(k)}$ (gen router)	Xavier	Uniform routing at init; prevents early specialization
$\boldsymbol{\phi}_g^{(k)}$ (gen bias)	Zero	No prior on generation assignment
$\mathbf{W}_f^{(g,k)}$ (flavor router)	Xavier	Uniform flavor selection at init
$\boldsymbol{\phi}_f^{(g,k)}$ (flavor bias)	Zero	No prior on flavor assignment
$\mathbf{W}_k$ (channel proj)	Xavier + ortho	Orthogonal init for channel diversity; QR decomposition
μ (budget dual var)	Zero	No budget penalty at start
budget_ema	Zero	No prior budget estimate

via QR decomposition and split it into K contiguous column blocks:

$$\mathbf{Q} = \mathrm{QR}(\mathbf{W}_{\text{rand}}), \quad \mathbf{W}_k = \mathbf{Q}\big[\,:,\ (k{-}1)d_{\text{proj}} : kd_{\text{proj}}\big], \quad k = 1, \ldots, K. \tag{68}$$

This ensures that the channel projections are orthonormal at initialization, providing a strong starting point for the orthogonality loss $\mathcal{L}_{\text{ortho}}$ to maintain. Without this, random projections may start nearly collinear in high dimensions, requiring many training steps to differentiate.

The **generation router initialization** uses Xavier (Glorot uniform) initialization rather than Kaiming because the router outputs a softmax distribution, not a ReLU activation. Kaiming initialization assumes half the neurons are zeroed by ReLU, which over-corrects the variance for softmax outputs. Xavier initialization maintains unit variance through the softmax, ensuring approximately uniform generation assignment at the start of training.

14.6 Training Pipeline: Visual Overview

Figure 5 provides a visual summary of the entire AS-MoE training pipeline, showing the three phases, the data flow between them, and which components are active at each stage.

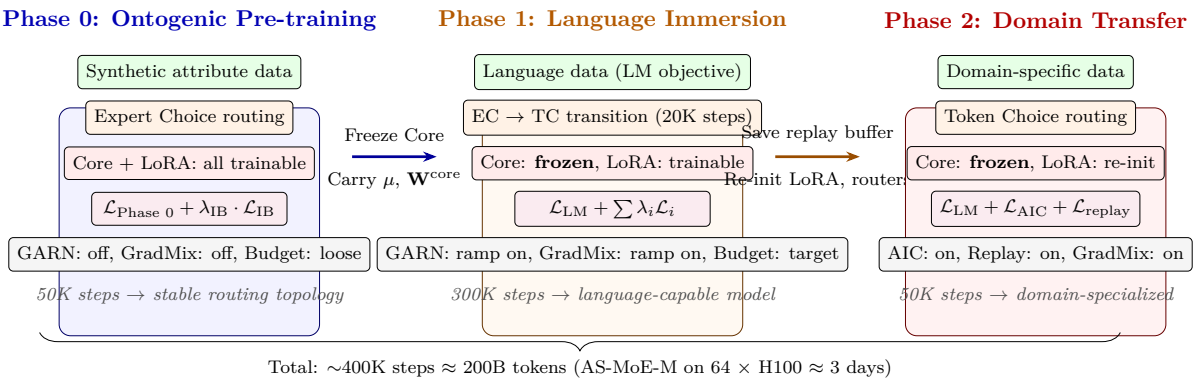

Figure 5: AS-MoE Training Pipeline. The three phases proceed left-to-right, with key components and data flowing between them. Phase 0 establishes routing topology; Phase 1 builds language competence; Phase 2 (optional) transfers to a new domain.

14.7 Training Loop: Complete Pseudocode

The following pseudocode integrates all components into a single training loop. This is the "runnable recipe" that a practitioner can translate directly into a training script. The loop handles all three phases, the component schedule, and the composite loss.

Input : Model $\mathcal{M}$ with AS-MoE layers, data loaders $\mathcal{D}_0, \mathcal{D}_1, \mathcal{D}_2$, hyperparams (Table 3)
Output: Trained model $\mathcal{M}^*$

```
// Phase 0: Ontogenic Pre-training
```
Freeze nothing; set $\gamma \leftarrow 0$, $\sigma_{\text{GARN}} \leftarrow 0$, $B_{\text{target}} \leftarrow 2.5$
$\alpha_{\text{EC}} \leftarrow 1.0$ `// Full Expert Choice`
for $t \leftarrow 1$ **to** T_0 **do**
  $\mathbf{x} \sim \mathcal{D}_0$ `// Synthetic attribute batch`
  $\hat{\mathbf{x}}, \{\mathcal{L}_{\text{aux}}^{(\ell)}\}_{\ell=1}^{L} \leftarrow \mathcal{M}.\text{forward}(\mathbf{x})$
  $\mathcal{L} \leftarrow \mathcal{L}_{\text{Phase 0}}(\hat{\mathbf{x}}, \mathbf{x}) + \sum_\ell \sum_j \lambda_j \mathcal{L}_{\text{aux},j}^{(\ell)}$
  $\mathcal{L}.\text{backward}();\ \text{clip_grad_norm}(\mathcal{M}, 1.0);\ \text{optimizer.step}()$
end
Checkpoint: save $\mathbf{W}^{\text{core}}$, μ, budget_ema

```
// Phase 1: Language Immersion
```
Freeze $\mathbf{W}^{\text{core}}$; set $\gamma \leftarrow 0.1$, $\sigma_{\text{GARN}} \leftarrow 0.1$, $B_{\text{target}} \leftarrow 2.0$
for $t \leftarrow 1$ **to** T_1 **do**
  $\alpha_{\text{EC}} \leftarrow \max(0, 1 - t/20000)$ `// EC → TC warmup`
  Ramp γ from 0 to 0.1 over first 5K steps
  Ramp σ_{GARN} from 0 to 0.1 over first 5K steps
  $\mathbf{x} \sim \mathcal{D}_1$ `// Language data batch`
  $\hat{\mathbf{x}}, \{\mathcal{L}_{\text{aux}}^{(\ell)}\}_{\ell=1}^{L} \leftarrow \mathcal{M}.\text{forward}(\mathbf{x})$
  $\mathcal{L} \leftarrow \mathcal{L}_{\text{LM}}(\hat{\mathbf{x}}) + \sum_\ell \sum_j \lambda_j \mathcal{L}_{\text{aux},j}^{(\ell)} + 0.1 \cdot \mathcal{L}_{\text{IB}}$
  $\mathcal{L}.\text{backward}();\ \text{clip_grad_norm}(\mathcal{M}, 1.0);\ \text{optimizer.step}()$
end
Checkpoint: save $\boldsymbol{\mu}_\pi, \boldsymbol{\Sigma}_\pi$ for replay buffer

```
// Phase 2: Domain Transfer (optional)
```
Freeze $\mathbf{W}^{\text{core}}$; re-init $\mathbf{A}, \mathbf{B}$, routers
Initialize $\mathcal{B}_{\text{replay}}$ from Phase 1 checkpoint
Compute $I_i^{(\text{act})}$ over 500 source-domain batches
for $t \leftarrow 1$ **to** T_2 **do**
  $\mathbf{x} \sim \mathcal{D}_2$ `// Domain-specific batch`
  $\hat{\mathbf{x}}, \{\mathcal{L}_{\text{aux}}^{(\ell)}\}, \pi^{(\ell)} \leftarrow \mathcal{M}.\text{forward}(\mathbf{x})$
  $\mathcal{L}_{\text{AIC}} \leftarrow \sum_{i \in \mathcal{I}_{g=0}} I_i^{(\text{act})} (\theta_i - \theta_i^{\text{pre}})^2$
  $\mathcal{L}_{\text{replay}} \leftarrow \mathcal{B}_{\text{replay}}.\text{compute_loss}(\pi^{(\ell)})$
  $\mathcal{L} \leftarrow \mathcal{L}_{\text{LM}}(\hat{\mathbf{x}}) + \sum_{\ell,j} \lambda_j \mathcal{L}_{\text{aux},j}^{(\ell)} + \lambda_{\text{AIC}} \mathcal{L}_{\text{AIC}} + \lambda_{\text{replay}} \mathcal{L}_{\text{replay}}$
  $\mathcal{L}.\text{backward}();\ \text{clip_grad_norm}(\mathcal{M}, 1.0);\ \text{optimizer.step}()$
end
return $\mathcal{M}$

Algorithm 4: AS-MoE Training Loop

The composite loss at each phase is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{main}} + \lambda_{\text{budget}} \mathcal{L}_{\text{budget}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}} + \lambda_{\text{ortho}} \mathcal{L}_{\text{ortho}} + \lambda_{\text{path}} \mathcal{L}_{\text{path}} + (\text{phase-specific terms}), \tag{69}$$

where $\mathcal{L}_{\text{main}}$ is $\mathcal{L}_{\text{Phase 0}}$ in Phase 0 and $\mathcal{L}_{\text{LM}}$ in Phases 1–2. The phase-specific terms are $\lambda_{\text{IB}} \mathcal{L}_{\text{IB}}$ in Phases 0–1, and $\lambda_{\text{AIC}} \mathcal{L}_{\text{AIC}} + \lambda_{\text{replay}} \mathcal{L}_{\text{replay}}$ in Phase 2. The auxiliary losses ($\mathcal{L}_{\text{budget}}$, $\mathcal{L}_{\text{cons}}$, $\mathcal{L}_{\text{div}}$, $\mathcal{L}_{\text{ortho}}$, $\mathcal{L}_{\text{path}}$) are computed per-layer and summed across all L layers.

14.8 Debugging Guide: Common Failure Modes and Remedies

AS-MoE introduces several interacting mechanisms (routing, budget control, gradient mixing, GARN) that can fail in non-obvious ways. This subsection catalogs the most common failure modes observed during development and provides concrete diagnostic and remedial procedures.

14.8.1 Routing Collapse

Symptom: One expert receives >80% of tokens across all layers; routing entropy drops below 0.5 bits; other experts receive negligible gradient signal.

Diagnosis: Compute the per-expert routing frequency $\bar{\pi}_j = \frac{1}{B \cdot T} \sum_{b,t} \pi_{b,t,j}$ and the routing entropy $H = -\sum_j \bar{\pi}_j \log \bar{\pi}_j$. Healthy routing has $H > \log(F) - 1$ (within 1 bit of uniform).

Remedies (in order of escalation):

1. **Increase GARN** $\sigma_{\mathbf{base}}$ from 0.1 to 0.2–0.3. GARN noise is the first line of defense against collapse, as it directly perturbs the routing distribution toward higher entropy.

2. **Increase diversity loss weight** $\lambda_{\mathbf{div}}$ from 0.01 to 0.05–0.1. This provides a stronger gradient signal pushing the flavor distribution toward uniform.

3. **Check generation router temperature** τ_g. If $\tau_g < 0.3$, the softmax is too peaked; increase to 0.5–1.0.

4. **Switch to Expert Choice temporarily**. If collapse occurs during Phase 1, pause the EC→TC transition and remain in Expert Choice mode for an additional 10K steps before resuming the transition.

5. **Re-initialize collapsed routers**. If a single router is consistently collapsed across multiple checkpoints, re-initialize its weights with Xavier and apply a 5K-step warmup with $\sigma_{\text{GARN}} = 0.3$.

14.8.2 Budget Drift

Symptom: The actual FLOP budget $\bar{\mathcal{B}}$ deviates by >10% from B_{target}, either exceeding it (expensive inference) or falling far below it (wasted capacity).

Diagnosis: Monitor $\bar{\mathcal{B}}$ and μ over training. Budget drift is typically caused by μ being too small (budget exceeds target) or too large (budget falls below target).

Remedies:

1. **Increase** η_μ (the dual variable step size). The current default is $\eta_\mu = 0.01$; increase to 0.05 for faster budget correction. This is safe because the EMA provides variance reduction.

2. **Decrease** $\beta_{\mathbf{EMA}}$ from 0.99 to 0.95. A lower EMA coefficient makes the budget estimate more responsive to recent changes, allowing μ to react faster to drift.

3. **Clamp generation assignments**. If budget consistently exceeds target, add a hard clamp: $g^* \leq g_{\max}$, where $g_{\max}$ is the largest generation such that $\sum_{g=0}^{g_{\max}} B^{(g)} \leq 1.1 \cdot B_{\text{target}}$.

14.8.3 Expert Stagnation (No Specialization)

Symptom: All experts in the same generation produce nearly identical outputs; the LoRA matrices **AB** remain close to zero; the model behaves like a single-expert model.

Diagnosis: Compute $\|\mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)}\|_F$ for each expert. If the Frobenius norm of all LoRA matrices remains below 10^{-3} after 10K training steps, experts are not differentiating.

Remedies:

1. **Increase LoRA learning rate**. The default LR for LoRA is 1×10^{-4} in Phase 1; increase to 3×10^{-4} or even 1×10^{-3} for the first 20K steps to encourage faster differentiation.

2. **Increase α/r scaling**. The LoRA scaling factor α/r controls the effective magnitude of the LoRA contribution. Increase α from 16 to 32 to allow larger deviations from the Core.

3. **Add LoRA dropout** (0.05–0.1) during Phase 0 to prevent co-adaptation of LoRA matrices across experts sharing the same Core.

14.8.4 Gradient Mixing Destabilizes Training

Symptom: Training loss oscillates or diverges after enabling gradient mixing; the mixing matrix $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$ has negative eigenvalues.

Diagnosis: Compute the eigenvalues of $\mathbf{R}$ (the cosine similarity matrix of expert outputs) per channel. If $\gamma \cdot \lambda_{\max}(\mathbf{R}) > 1$, the mixing matrix $\mathbf{M}$ has negative eigenvalues, causing gradient sign reversal.

Remedy: Reduce γ to satisfy the Gershgorin condition $\gamma < 1/\max_i \sum_{j \neq i} |R_{ij}|$. In practice, $\gamma \leq 0.1$ is safe for $N_e \leq 24$ experts; for larger models, use $\gamma \leq 0.05$.

14.8.5 Channel Collapse

Symptom: All channel projections $\mathbf{W}_k$ converge to similar subspaces; $\mathcal{L}_{\text{ortho}}$ drops to near zero but channels produce correlated outputs.

Diagnosis: Compute the effective rank of the stacked projections $[\mathbf{W}_1; \ldots; \mathbf{W}_K]$ and the pairwise cosine similarity between channels. If the cosine similarity exceeds 0.8, channels are collapsed.

Remedies:

1. **Increase λ_{ortho}** from 0.1 to 0.5–1.0.

2. **Re-initialize channel projections** using the QR-based orthogonal initialization described in Section 14.5.

3. **Use different random seeds** for each channel projection during initialization, rather than splitting a single QR matrix.

15 Reference Implementation

15.1 GradientMixing (Dimension-Preserving, Per-Channel)

The gradient mixing autograd function preserves full tensor dimensions in the backward pass, applying the correlation-based mixing via `einsum`. As established in Remark 8.1, mixing is applied independently within each channel over the $N = G \times F$ experts of that channel:

```
class GradientMixing(Function):
    @staticmethod
    def forward(ctx, expert_stack, weights, gamma):
        # expert_stack: [N, B, T, D] - stacked expert
        #   outputs within ONE channel (N = G*F)
        # weights: [B, T, N] - routing weights
        # gamma: mixing strength
        ctx.save_for_backward(
            expert_stack.bfloat16(),
            weights.bfloat16(),
            torch.tensor(gamma))
        ctx.gamma = gamma
        # Weighted sum: [B, T, D]
        return torch.einsum('bti,ibtd->btd', weights,
                            expert_stack.float())
```

```python

    @staticmethod
    def backward(ctx, grad_output):
        # grad_output: [B, T, D]
        expert_stack, weights, gamma_t = ctx.saved_tensors
        gamma = gamma_t.item()
        N = expert_stack.shape[0]

        # Compute correlation matrix R: [N, N]
        flat = expert_stack.float().reshape(N, -1)  # [N, B*T*D]
        R = torch.nn.functional.cosine_similarity(
            flat.unsqueeze(1), flat.unsqueeze(0), dim=-1)
        M = torch.eye(N, device=grad_output.device) + gamma * R

        # Raw gradients: [N, B, T, D]
        raw = torch.einsum('btd,bti->ibtd',
                           grad_output, weights.float())
        # Mixed gradients via einsum (preserves dims):
        # mixed_i = sum_j M[j,i] * raw_j
        mixed = torch.einsum('ji,jbtd->ibtd', M, raw)

        return mixed, None, None
```

15.2 ASMoELayer (Vectorized Forward)

The following implementation eliminates Python loops over batch and sequence dimensions, using vectorized operations and batched GEMM:

```python
class ASMoELayer(nn.Module):
    def __init__(
        self, d_model, G=3, F=4, K=2,
        d_ffs=[1024, 2048, 4096],
        budgets=[1.0, 2.0, 4.0],
        lora_r=8, lora_alpha=16.0,
        gamma_mix=0.1, target_budget=2.0,
        beta_ema=0.99, eta_mu=0.01,
        top_m=1, d_proj=None,
        garn_sigma_base=0.1, garn_eta=2.0,
        garn_alpha=0.5, gen_temp=1.0
    ):
        super().__init__()
        self.G, self.F, self.K = G, F, K
        self.gamma_mix = gamma_mix
        self.target_budget = target_budget
        self.budgets_val = budgets
        self.d_ffs = d_ffs
        self.lora_r = lora_r
        self.lora_alpha = lora_alpha
        self.top_m = top_m
        self.garn_sigma_base = garn_sigma_base
        self.garn_eta = garn_eta
        self.garn_alpha = garn_alpha
        d_proj = d_proj or d_model // 2

        # Channel projections (multi-view)
        self.projs = nn.ParameterList([
            nn.Parameter(torch.randn(d_model, d_proj)
                         * (2.0/d_model)**0.5)
            for _ in range(K)])

        # Generation routers (one per channel, consensus)
        self.gen_routers = nn.ModuleList([
            nn.Linear(d_proj, G) for _ in range(K)])

        # Flavor routers: per (generation, channel)
        # W_f^{(g,k)} as in Definition 5.2
        self.flavor_routers = nn.ModuleDict({
            f"{g}_{k}": nn.Linear(d_proj, F)
            for g in range(G) for k in range(K)})

        # Expert params: Core + LoRA (same as before)
```

```python
        self.cores_up = nn.ParameterList()
        self.cores_down = nn.ParameterList()
        self.lora_A_up = nn.ParameterList()
        self.lora_B_up = nn.ParameterList()
        self.lora_A_down = nn.ParameterList()
        self.lora_B_down = nn.ParameterList()
        for g in range(G):
            d_ff = d_ffs[g]
            self.cores_up.append(
                nn.Parameter(torch.randn(d_model, d_ff)
                             * (2.0/d_model)**0.5))
            self.cores_down.append(
                nn.Parameter(torch.randn(d_ff, d_model)
                             * (2.0/d_ff)**0.5))
            for f in range(F):
                for k in range(K):
                    self.lora_A_up.append(
                        nn.Parameter(torch.randn(
                            d_model, lora_r)
                            * (2.0/d_model)**0.5))
                    self.lora_B_up.append(
                        nn.Parameter(torch.zeros(
                            lora_r, d_ff)))
                    self.lora_A_down.append(
                        nn.Parameter(torch.randn(
                            d_ff, lora_r)
                            * (2.0/d_ff)**0.5))
                    self.lora_B_down.append(
                        nn.Parameter(torch.zeros(
                            lora_r, d_model)))

        # Budget controller: separate EMA and dual var
        self.register_buffer(
            'budget_ema', torch.tensor(0.0))
        self.register_buffer(
            'mu', torch.tensor(0.0))
        self.beta_ema = beta_ema
        self.eta_mu = eta_mu
        self.gen_temp = gen_temp

    def _lora_idx(self, g, f, k):
        return g * self.F * self.K + f * self.K + k

    def _garn_noise(self, logits, probs):
        """GARN: entropy-adaptive + gradient-adaptive noise.
        NOTE: The gradient-adaptive term uses |logits| as a
        proxy for |grad_z L|, since true logit gradients are
        unavailable in forward pass. This is justified by the
        correlation between logit magnitude and softmax-gradient
        magnitude: d L/d z_i = pi_i (delta_{i,y} - pi_y),
        which is proportional to pi_i ~ exp(z_i) for the
        dominant expert. For a true gradient-adaptive term,
        a two-pass forward (first pass without noise, then
        backward to obtain grad_z, then second pass with
        gradient-scaled noise) would be needed at 2x cost.
        """
        H = -(probs * (probs + 1e-8).log()).sum(-1)
        sigma = self.garn_sigma_base * torch.exp(
            -self.garn_eta * H)
        # Gradient-adaptive term (proxy: logit magnitude
        # correlates with |grad_z L| via softmax Jacobian)
        sigma = sigma.unsqueeze(-1) * (
            1 + self.garn_alpha * logits.abs())
        return sigma * torch.randn_like(logits)

    def forward(self, x):
        B, T, D = x.shape
        ch_outputs = []
        gen_dists = []  # for consensus loss
        flv_dists = []  # for diversity loss

        for k in range(self.K):
            h = x @ self.projs[k]  # (B, T, d_proj)
```

```python

            # --- Generation routing with GARN ---
            gen_logits = self.gen_routers[k](h)
            if self.training:
                gen_logits = gen_logits + self._garn_noise(
                    gen_logits,
                    F.softmax(gen_logits / self.gen_temp,
                              dim=-1))
            gen_probs = F.softmax(
                gen_logits / self.gen_temp, dim=-1)

            # STE: hard forward, soft backward
            g_hard = F.one_hot(
                gen_probs.argmax(-1), self.G
            ).float()
            g_hard = (g_hard
                      + gen_probs - gen_probs.detach())

            gen_dists.append(gen_probs)

            # --- Flavor routing with GARN ---
            ch_expert_outs = []
            ch_weights = []
            for g in range(self.G):
                flv_logits = self.flavor_routers[
                    f"{g}_{k}"](h)
                if self.training:
                    flv_logits = flv_logits + self._garn_noise(
                        flv_logits,
                        F.softmax(flv_logits, dim=-1))
                flv_probs = F.softmax(flv_logits, dim=-1)
                flv_dists.append(flv_probs)

                # Top-m flavor selection
                top_vals, top_idx = flv_probs.topk(
                    self.top_m, dim=-1)
                # One-hot for selected flavors
                flv_mask = F.one_hot(
                    top_idx, self.F
                ).float().sum(dim=-2)  # (B, T, F)
                flv_mask = (flv_mask
                            + flv_probs - flv_probs.detach())

                # Gather tokens for this generation
                g_mask = (gen_probs.argmax(-1) >= g
                          ).float()  # curriculum
                tokens_g = x  # process all, mask later

                for f in range(self.F):
                    idx = self._lora_idx(g, f, k)
                    scale = self.lora_alpha / self.lora_r
                    W_up = (self.cores_up[g]
                            + scale * self.lora_A_up[idx]
                            @ self.lora_B_up[idx])
                    W_down = (self.cores_down[g]
                              + scale * self.lora_A_down[idx]
                              @ self.lora_B_down[idx])
                    hidden = F.gelu(tokens_g @ W_up)
                    out_f = hidden @ W_down  # (B, T, D)
                    ch_expert_outs.append(out_f)
                    # Weight = flv_mask for f * g_mask
                    w = (flv_mask[..., f:f+1]
                         * g_mask.unsqueeze(-1))
                    ch_weights.append(w)

            # Stack and apply gradient mixing
            expert_stack = torch.stack(
                ch_expert_outs, dim=0)  # [N, B, T, D]
            weight_stack = torch.stack(
                ch_weights, dim=-1)      # [B, T, N]
            if self.training:
                ch_out = GradientMixing.apply(
                    expert_stack, weight_stack,
```

```
                    self.gamma_mix)
            else:
                ch_out = torch.einsum(
                    'bti,ibtd->btd',
                    weight_stack, expert_stack)
            ch_outputs.append(ch_out)

        # Average over channels (no curriculum norm)
        final = sum(ch_outputs) / self.K

        # --- Budget: actual FLOPs from hard routing ---
        # Curriculum cost: a token assigned to generation g
        # receives contributions from ALL generations <= g.
        # So actual_cost = sum_g P(g_assign >= g) * B^(g),
        # which correctly accounts for the cumulative budget.
        g_assign = gen_dists[0].argmax(-1)  # (B, T)
        actual_cost = sum(
            (g_assign >= g).float().mean()
            * self.budgets_val[g]
            for g in range(self.G))

        # Two-phase budget update (Lagrangian dual ascent):
        # Phase A: EMA for variance-reduced budget estimate
        self.budget_ema.copy_(
            self.beta_ema * self.budget_ema
            + (1 - self.beta_ema) * actual_cost)
        # Phase B: gradient step on mu (using PRE-update mu
        # for budget_loss, then update mu for next step).
        # NOTE: budget_loss uses old mu; mu is updated
        # simultaneously with theta. This creates a one-step
        # lag, which is acceptable in practice but differs
        # from strict primal-dual (which updates mu AFTER
        # the theta gradient step). For strict primal-dual,
        # move mu.update() outside the forward pass.
        budget_loss = self.mu * (
            self.budget_ema - self.target_budget)
        self.mu.copy_(torch.clamp(
            self.mu + self.eta_mu * (
                self.budget_ema - self.target_budget),
            min=0.0))

        # --- Consensus loss (cross-channel) ---
        cons_loss = -sum(
            F.cosine_similarity(
                gen_dists[i], gen_dists[j], dim=-1
            ).mean()
            for i in range(self.K)
            for j in range(i+1, self.K))

        # --- Diversity loss (per-generation averaging) ---
        # Definition 5.2: div is averaged per (g,k) pair,
        # not across all generations simultaneously.
        div_loss = sum(
            F.mse_loss(
                flv.mean(dim=[0,1]),  # avg over B,T
                torch.ones(self.F, device=x.device) / self.F)
            for flv in flv_dists
        ) / len(flv_dists)

        # --- Channel orthogonality loss ---
        ortho_loss = sum(
            torch.norm(
                self.projs[i].T @ self.projs[j], p='fro')**2
            for i in range(self.K)
            for j in range(i+1, self.K))

        return final, {
            'budget': budget_loss,
            'consensus': cons_loss,
            'diversity': div_loss,
            'ortho': ortho_loss,
        }
```

15.3 Routing Replay Buffer

```python
class RoutingReplayBuffer:
    """Stores mean and covariance of routing distributions
    from source domain for anti-forgetting during transfer."""

    def __init__(self, num_layers, num_experts,
                 device='cuda'):
        self.num_layers = num_layers
        self.num_experts = num_experts
        self.device = device
        # Store on GPU for in-place lerp_ updates
        self.means = [torch.zeros(num_experts,
                      device=device)
                      for _ in range(num_layers)]
        self.covs = [torch.eye(num_experts,
                     device=device)
                     for _ in range(num_layers)]
        self.count = 0

    @torch.no_grad()
    def update(self, routing_distributions):
        """Update buffer with new routing distributions.
        Args:
            routing_distributions: list of (B*T, N_e) tensors,
                one per layer.
        """
        self.count += 1
        alpha = min(0.1, 1.0 / self.count)
        for l, pi in enumerate(routing_distributions):
            mean_new = pi.mean(dim=0)
            cov_new = torch.cov(pi.T)
            # In-place EMA update (avoids CPU<->GPU copies)
            self.means[l].lerp_(mean_new, alpha)
            self.covs[l].lerp_(cov_new, alpha)

    def compute_loss(self, current_distributions, lam_sigma=0.1):
        """Compute replay loss against stored statistics."""
        loss = 0.0
        for l, pi in enumerate(current_distributions):
            mean_curr = pi.mean(dim=0)
            cov_curr = torch.cov(pi.T)
            loss += F.mse_loss(mean_curr, self.means[l])
            loss += lam_sigma * F.mse_loss(
                cov_curr, self.covs[l])
        return loss / self.num_layers
```

16 Experimental Design

16.1 Datasets and Benchmarks

To rigorously evaluate the hierarchical and compositional capabilities of AS-MoE, we structure our benchmarks across the three training phases:

Phase 0: Ontogenic Pre-training. We utilize a procedurally generated synthetic dataset comprising discrete attributes (e.g., entity type, relation, syntactic role). The dataset consists of 10^8 synthetic tokens structurally designed to enforce compositional relationships, allowing us to explicitly measure the attribute-to-flavor alignment prior to language modeling.

Phase 1: Language Immersion. For natural language pre-training, we use a 1-Trillion token subset of a standard high-quality corpus (e.g., SlimPajama or RefinedWeb). Evaluation is conducted on standard zero-shot and few-shot benchmarks to ensure the Core+LoRA structure matches dense or standard MoE baselines:

- **General Knowledge**: MMLU, ARC-Challenge, Hellaswag.
- **Reasoning**: PIQA, WinoGrande.

Phase 2: Domain Transfer. To evaluate the Core+LoRA factorization and the Routing Replay Buffer, we perform domain transfer on distinct distributions:

- **Code Generation**: HumanEval and MBPP, fine-tuned on the StarCoder corpus.
- **Mathematics**: GSM8K and MATH, fine-tuned on OpenWebMath.

16.2 Proposed Benchmarks

We propose the following experimental configurations to validate AS-MoE:

Table 6: Proposed experimental configurations for AS-MoE validation.

Config	Params	G	F	K
AS-MoE-S	1.3B	2	4	2
AS-MoE-M	6.7B	3	4	2
AS-MoE-L	13B	3	8	4

16.3 Routing Stability and Utilization Metrics

Beyond standard language modeling perplexity, the efficacy of our structural constraints (GARN, Gradient Mixing, Path Consensus) is measured using specific topological metrics:

Definition 16.1 (Expert Load Variance)**.** To measure load balance efficiency without relying on Expert Choice routing during inference, we track the coefficient of variation (CV) of the assigned token counts c_i across all N_e experts:

$$\mathrm{CV}_{\mathrm{load}} = \frac{\sigma_c}{\mu_c} = \frac{\sqrt{\frac{1}{N_e}\sum_{i=1}^{N_e}(c_i - \bar{c})^2}}{\bar{c}}. \tag{70}$$

A lower $\mathrm{CV}_{\mathrm{load}}$ indicates superior utilization of the expert ensemble.

Definition 16.2 (Representation Collapse Margin)**.** We quantify router collapse by monitoring the average routing entropy over a batch B:

$$\bar{H}_{\mathrm{route}} = \frac{1}{|B|}\sum_{\mathbf{x}\in B}\left(-\sum_{i=1}^{N_e}\pi_i(\mathbf{x})\log\pi_i(\mathbf{x})\right). \tag{71}$$

A sudden plunge in $\bar{H}_{\mathrm{route}}$ approaching 0 signifies mode collapse. The GARN mechanism (Section 9) is actively tuned to prevent $\bar{H}_{\mathrm{route}}$ from crossing a predefined critical threshold. The Fokker–Planck analysis (Proposition 9.2) provides the theoretical guarantee that the stationary distribution has zero mass at the collapse boundary.

Definition 16.3 (Path Coherence Score)**.** To validate the Path Consensus Loss (Section 10), we define the coherence score $\mathcal{C}_{\mathrm{path}}$ as the average overlap of the top-m selected experts across adjacent layers ℓ and $\ell+1$:

$$\mathcal{C}_{\mathrm{path}} = \mathbb{E}_{\mathbf{x}\sim p_{\mathrm{data}}}\left[\frac{|\mathcal{S}^{(\ell)}(\mathbf{x})\cap\mathcal{S}^{(\ell+1)}(\mathbf{x})|}{m}\right] \in [0,1]. \tag{72}$$

16.4 Ablation Studies

The following ablation studies are proposed to isolate the contribution of each architectural component:

1. **Generation hierarchy**: Remove the curriculum logic (only use the selected generation, not all $\leq g$).
2. **Gradient mixing**: Set $\gamma = 0$ (no mixing) and compare training stability and expert utilization.
3. **GARN**: Replace GARN with fixed Gaussian noise ($\sigma = \text{const}$) and measure router collapse rates.
4. **Path consensus**: Set $\lambda_{\text{path}} = 0$ and measure cross-layer routing consistency.
5. **Ontogenic pre-training**: Train from scratch on language data (skip Phase 0) and compare sample efficiency.
6. **Core + LoRA**: Replace with full-rank expert weights (no factorization) and compare transfer performance.
7. **Hybrid routing**: Use only Token Choice throughout training and compare with the EC $\rightarrow$ TC transition.

17 Discussion

17.1 From Quark Analogy to Standard Model: From Metaphor to Method

The development of AS-MoE illustrates a broader pattern in architecture design: the transition from informal analogy to formal theory. The original quark model analogy—with its "generations" (up/down, strange/charm, top/bottom), "flavors" (functional specialization), and "colors" (multi-view representation)—provided an intuitive organizational framework. However, each component has been independently justified through engineering necessity, and the analogy has since expanded to cover additional particles of the Standard Model:

- **Generation hierarchy** is justified by the observation that not all tokens require the same computational depth (a fact established by early-exit and adaptive computation research).
- **Flavor specialization** is justified by the success of shared experts (DeepSeek) and the failure mode of unstructured routing (expert redundancy).
- **Multi-view channels** are justified by the multi-head latent attention (MLA) principle: independent projections capture complementary aspects of the input.
- **Confinement** (experts must cooperate, not isolate) is justified by the empirical observation of mode collapse in MoE training and the theoretical analysis via gradient mixing (Lemma 8.1).
- **Gluon exchange** (Remark 8.2) is justified by the $\gamma\mathbf{R}$ term in gradient mixing: the off-diagonal correlations mediate inter-expert gradient transfer, exactly as gluons mediate the strong force. The Gershgorin bound on γ corresponds to the asymptotic freedom regime of QCD.

- **Higgs vacuum** (Remark 5.1) is justified by the role of Core weights as a nonzero background that breaks the symmetry between experts at initialization ($\mathbf{B} = 0$), giving each expert a "mass" (FLOP budget) through the generation structure. The Kato perturbation bound ensures that LoRA excitations do not destabilize this vacuum.
- **Weak interaction and CKM matrix** (Remark 6.1) is justified by the generation transition logic: the curriculum constraint enforces upward-only transitions analogous to W-boson-mediated flavor change, and the routing transition matrix $\mathbf{V}$ plays the role of the CKM matrix, quantifying inter-generation mixing rates controlled by the Lagrangian dual variable μ.

The Standard Model analogy thus provides a *complete* mapping from particle physics to AS-MoE architecture: quarks (expert topology), gluons (gradient mixing), the Higgs field (Core weights), and weak bosons (generation transitions) each correspond to a distinct architectural mechanism. Notably, leptons and photons have *no* natural counterpart in AS-MoE: leptons lack color charge and would correspond to non-expert components (e.g., MLA attention heads) that are outside the MoE layer, while photons (long-range electromagnetic force) have no obvious MoE analogue. The incompleteness of the mapping is itself informative—it demarcates precisely which aspects of the architecture the particle physics analogy can illuminate and where it must give way to independent engineering justification.

17.2 Known Implementation Issues

The reference implementation (Section 15) addresses the major code-text discrepancies identified during review, but several issues remain:

1. **Expert Choice → Token Choice transition**: The hybrid routing strategy (Section 6) requires a distillation loss to smooth the EC→TC transition. This loss is not yet implemented and is deferred to the Triton kernel implementation.
2. **Reference implementation is not truly sparse**: The current forward pass (Section 15) processes *all* tokens through *all* experts and then masks outputs by routing weights. This means the FLOP cost is $O(B \cdot T \cdot G \cdot F \cdot K \cdot D \cdot d_{\text{ff}})$—full dense compute—rather than the sparse $O(B \cdot T \cdot N_{\text{active}} \cdot D \cdot d_{\text{ff}})$ described in the architecture. The reference implementation demonstrates *correctness* of the routing and mixing logic, but **not efficiency**. True sparse execution requires custom CUDA/Triton kernels with block scheduling that only launch expert computation for routed tokens.
3. **GARN gradient-adaptive term uses logit proxy**: As discussed in Remark 9.1, the gradient term $|\nabla_{z_i}\mathcal{L}|$ in Definition 9.1 is approximated by $|z_i|$ in the reference implementation, since true logit gradients are unavailable during the forward pass. This approximation preserves qualitative behavior but introduces bias. A two-pass GARN would be more accurate at $2\times$ cost.
4. **Full vectorization requires custom CUDA**: The current implementation eliminates Python loops over batch/sequence but still iterates over experts within a channel. A fully fused implementation requires custom CUDA/Triton kernels with block scheduling (see Section 13).
5. **SoA layout not yet realized in PyTorch**: The SoA memory layout described in Section 13 is conceptual. The PyTorch reference uses standard `nn.ParameterList` (AoS). Production deployment requires explicit SoA tensors with expert offsets per generation:

```
# SoA layout for production
self.w_up = nn.Parameter(torch.empty(
    G, F, K, d_model, d_ff_max))
# + expert_offsets for varying d_ff per g
```

6. **Gradient mixing across DDP**: The all-gather for global correlation matrix computation is described but not implemented. This requires a custom communication hook in PyTorch DDP.

7. **Simultaneous vs. sequential dual update**: The budget controller updates μ simultaneously with the forward pass, creating a one-step lag between budget violation and penalty (see Section 15 for the detailed comment). Strict primal-dual methods update μ *after* the primal gradient step. This is acceptable in practice but theoretically imprecise.

17.3 Limitations and Open Questions

Several open questions remain:

1. **Scalability**: The current analysis assumes a fixed number of experts per layer. Scaling to millions of experts (as in some proposed architectures) requires different routing mechanisms and systems design.

2. **Dynamic** G, F, K: The current architecture uses fixed topology hyperparameters. Adaptive topology—where the number of active flavors or channels varies per layer or during training—is an unexplored direction.

3. **Optimal LoRA rank**: The choice of LoRA rank r is currently a hyperparameter. Theory from Kato perturbation analysis (Proposition 11.2) suggests that r should scale with the spectral gap of the Core operator. This connection has been formalized through Theorem 11.3 (Spectral Isolation and Subspace Preservation): if δ is the spectral gap of $\mathbf{W}^{\text{core}}$ at rank r, then the LoRA perturbation must satisfy $\|\Delta\mathbf{W}\|_2 < \delta/2$ to prevent spectral bifurcation. The optimal rank r is determined by the index of the maximal spectral gap in the Core SVD (see Remark 11.2).

4. **Functor approximation**: The functor $\mathcal{F}$ (Theorem 12.1) is approximated in practice by the inductive bias loss. Characterizing the approximation error and its impact on downstream performance is an open theoretical question. The sample complexity of ontogenic pre-training has been bounded by Theorem 12.2: the number of synthetic quark samples needed for ϵ-approximation of $\mathcal{F}$ depends on the VC-dimension of the routing network $d_{\text{VC}}(\mathcal{H}_{\text{route}}) = \mathcal{O}(N_e \cdot d_{\text{proj}} \log d_{\text{proj}})$, which is independent of the full model dimension and thus exponentially smaller than end-to-end learning (see Remark 12.1). The domain transfer process has been further formalized through natural transformations (Section 12), providing a category-theoretic explanation for why Core + LoRA transfers efficiently.

5. **Triton kernel optimization**: The current implementation uses PyTorch with conceptual Grouped GEMM. A production implementation requires custom Triton kernels with block scheduling and GARN integration, which is deferred to future work.

6. **Aggregation strategy for ontogenic embeddings**: The quark aggregation operator $\oplus$ (Section 12) currently uses mean pooling. Attention-weighted aggregation may better capture attribute interactions but introduces additional parameters.

7. **Ablation scale equivalence**: In the proposed ablation studies (Section 16), AS-MoE-S (1.3B total parameters) should be compared against dense models with equivalent *active* parameter counts, not equivalent *total* parameter counts. The equivalent dense baseline for AS-MoE-S with target budget 2.0 is approximately 0.8B parameters.

8. **Communication lower bound for fused computation-communication**: For the fused compute-communicate pattern (Section 13), we conjecture that latency $\geq \max(T_{\text{comp}}, T_{\text{comm}})$ and that prefetching achieves this lower bound when $T_{\text{comp}} \geq T_{\text{comm}}$. A formal proof would establish the optimality of the NVSHMEM-based pipeline.

18 Conclusion

We have presented AS-MoE, an architecture that imposes principled topological, algebraic, and dynamical structure on Mixture of Experts. By organizing experts along three orthogonal axes (Generation, Flavor, Channel), factorizing parameters into Core + LoRA components with Kato perturbation-theoretic justification (the Higgs vacuum analogy, formalized through the Ginzburg–Landau potential for spontaneous symmetry breaking), enforcing FLOP budgets via Lagrangian duality with two-phase dual updates (justified by Borkar's two-time-scale stochastic approximation), preventing expert isolation through covariance gradient mixing with Gershgorin convergence guarantees (the gluon exchange analogy, formalized through the gauge-theoretic covariant derivative and the asymptotic freedom trade-off), modeling generation transitions through a CKM-like routing matrix (the weak interaction analogy), stabilizing routing via GARN with Fokker–Planck guarantees of boundary repulsion and collapse prevention, ensuring cross-layer coherence via Path Consensus, and formalizing ontogenic pre-training through category-theoretic functors and natural transformations, AS-MoE provides a comprehensive, formally grounded framework for structured MoE design. The optimal LoRA rank is determined by the spectral gap of the Core operator via the Davis–Kahan theorem (Theorem 11.3), and the sample efficiency of ontogenic pre-training is bounded by the VC-dimension of the routing network (Theorem 12.2). The Standard Model analogy—quarks (topology), gluons (mixing), Higgs (factorization), and weak bosons (routing)—provides a productive organizing language, but every architectural decision is independently justified on engineering grounds. The systems design leverages Data-Oriented Design principles, fused Grouped GEMM, persistent kernels, and NVSHMEM for efficient GPU execution. We believe this work represents a step toward MoE architectures that are not only scalable and efficient, but also theoretically principled and predictably behaved.

A Engineering PRD: AS-MoE Production System

This appendix provides a Product Requirements Document (PRD) for a production AS-MoE system, translating the architectural theory into engineering specifications.

A.1 Overview

The AS-MoE Production System is a distributed training and inference framework for the Adaptive Structured Mixture of Experts architecture. It supports the full training lifecycle: ontogenic pre-training (Phase 0), language immersion (Phase 1), and domain transfer (Phase 2), with production inference serving.

A.2 System Requirements

A.3 Architecture Components

A.3.1 Training Pipeline

A.3.2 Inference Pipeline

Table 7: AS-MoE system requirements matrix.

ID	Requirement	Priority
SR-01	Support models up to 100B total parameters (13B active per token)	P0
SR-02	Training throughput $\geq$ 4000 tokens/sec/GPU on H100	P0
SR-03	Inference latency $\leq$ 30ms/token on single H100	P0
SR-04	Budget adherence: actual FLOPs within 5% of target	P0
SR-05	Support Expert Choice $\rightarrow$ Token Choice routing transition	P1
SR-06	Gradient mixing with DDP all-gather	P1
SR-07	NVSHMEM-based expert prefetching	P1
SR-08	Routing replay buffer for transfer learning	P2
SR-09	Dynamic topology (adaptive F, K)	P2
SR-10	Triton kernel for fused Grouped GEMM + GARN	P2

Table 8: Training pipeline components.

Component	Language	Description
ASMoELayer	Python/PyTorch	Core MoE layer with Core+LoRA, multi-channel routing, gradient mixing
BudgetController	Python/PyTorch	Lagrangian dual variable μ with EMA
GARNModule	Python/PyTorch	Gradient-adaptive router noise
RoutingReplayBuffer	Python/PyTorch	Mean/cov storage for transfer
FusedASMoEKernal	Triton	Fused Grouped GEMM + activation + GARN
CommOverlay	C++/CUDA	NVSHMEM-based expert prefetching

Table 9: Inference pipeline components.

Component	Language	Description
TokenChoiceRouter	C++/CUDA	Low-latency top-m routing with KV-cache
ExpertCache	C++	LRU cache for LoRA adapters
BatchScheduler	C++	Dynamic batching across generation levels

A.4 Configuration Schema

The default configuration for AS-MoE-M (6.7B parameters):

```python
# AS-MoE-M default configuration
config = {
    "model": {
        "d_model": 4096,
        "n_layers": 32,
        "n_heads": 32,
        "mla_latent_dim": 512,
        "vocab_size": 32000,
    },
    "moe": {
        "G": 3,        # generations
        "F": 4,        # flavors
        "K": 2,        # channels
        "d_ffs": [1024, 2048, 4096],
        "budgets": [1.0, 2.0, 4.0],
        "lora_r": 8,
        "lora_alpha": 16.0,
        "top_m": 1,
        "d_proj": 2048,
    },
    "training": {
        "gamma_mix": 0.1,
        "target_budget": 2.0,
        "beta_ema": 0.99,
        "garn_sigma_base": 0.1,
        "garn_eta": 2.0,
        "garn_alpha": 0.5,
        "lambda_path": 0.01,
        "lambda_ewc": 1000.0,
        "lambda_replay": 0.1,
    },
    "system": {
        "dtype": "bfloat16",
        "kernel_fusion": True,
        "nvshmem_prefetch": True,
        "persistent_kernel": True,
    }
}
```

A.5 Milestones

Table 10: AS-MoE development milestones.

Phase	Timeline	Deliverable
M1	Week 1–4	PyTorch reference implementation (SR-01, SR-04)
M2	Week 5–8	Distributed training with DDP (SR-02, SR-06)
M3	Week 9–12	Triton kernel + inference optimization (SR-03, SR-10)
M4	Week 13–16	NVSHMEM integration + EC→TC transition (SR-05, SR-07)
M5	Week 17–20	Transfer learning pipeline (SR-08) + evaluation

B Notation Reference

Table 11: Summary of notation used throughout this paper.

Symbol	Domain	Description
G	$\mathbb{N}$	Number of generations
F	$\mathbb{N}$	Number of flavors
K	$\mathbb{N}$	Number of channels (views)
g, f, k	—	Generation, flavor, channel indices
d_{model}	$\mathbb{N}$	Model hidden dimension
$d_{\text{ff}}^{(g)}$	$\mathbb{N}$	FFN intermediate dimension for generation g
$\mathbf{W}^{\text{core}}$	$\mathbb{R}^{d\times d'}$	Core weight matrix (shared, frozen)
$\mathbf{A}, \mathbf{B}$	$\mathbb{R}^{d\times r}$, $\mathbb{R}^{r\times d'}$	LoRA adaptation matrices
r	$\mathbb{N}$	LoRA rank
α	$\mathbb{R}^+$	LoRA scaling factor
$\pi_g, \pi_f^{(k)}$	Δ^{G-1}, Δ^{F-1}	Routing distributions
$\mathbf{M}$	$\mathbb{R}^{N\times N}$	Gradient mixing matrix
$\mathbf{R}$	$\mathbb{R}^{N\times N}$	Activation correlation matrix
γ	$\mathbb{R}^+$	Gradient mixing strength
μ	$\mathbb{R}^+$	Lagrangian dual variable (budget)
β	$(0, 1)$	EMA decay rate
σ_{GARN}	$\mathbb{R}^+$	GARN noise scale
$H(\pi)$	$\mathbb{R}^+$	Entropy of routing distribution
$\mathcal{F}$	Functor	Compositional bridge $\mathbf{C}_{\text{quark}} \to \mathbf{C}_{\text{lang}}$

References

[1] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.

[2] M. I. Jordan and R. A. Jacobs, "Hierarchical mixtures of experts and the EM algorithm," *Neural Computation*, vol. 6, no. 2, pp. 181–214, 1994.

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[4] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.

[5] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[6] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. Dai, Z. Chen, Q. Le, and J. Wei, "Mixture-of-experts with expert choice routing," in *Proc. NeurIPS*, 2022.

[7] D. Dai et al., "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[8] DeepSeek-AI et al., "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[10] T. Gale, D. Elsen, and S. Hooker, "The state of sparse training in deep learning," in *Proc. ICML*, 2023.

[11] L. Arnold, *Random Dynamical Systems*. Springer, 2013.

[12] T. Kato, *Perturbation Theory for Linear Operators*. Springer, 2nd ed., 1980.

[13] B. Zoph, W. Fedus, L. Zettlemoyer, B. Li, J. Wei, and N. Shazeer, "ST-MoE: Designing stable and transferable sparse expert models," *arXiv preprint arXiv:2209.14589*, 2022.

[14] M. Lewis, S. Ghosh, and T. Liu, "BASE layers: Simplifying training of large, sparse models," in *Proc. ICML*, 2021.

[15] F. Wang et al., "IniLoRA: On the initialization of LoRA in large language models," *arXiv preprint*, 2024.

[16] Q. Shen et al., "PathMoE: Cross-layer routing for mixture-of-experts," *arXiv preprint*, 2024.

[17] H. Zhao et al., "FlashDMoE: Fast distributed MoE with persistent kernels," *arXiv preprint*, 2024.

[18] S. Kim et al., "SonicMoE: Enabling large-scale distributed MoE training with optimized kernel design," *arXiv preprint*, 2024.

[19] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *PNAS*, vol. 114, no. 13, pp. 3521–3526, 2017.

[20] P. Awasthi et al., "From sparse to structured: Rethinking MoE organization," *arXiv preprint*, 2024.

[21] S. Mac Lane, *Categories for the Working Mathematician*. Springer, 2nd ed., 1998.